SUP'COM

المدرسة العليا للمواصلات بتونس
ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Telecommunications Engineering Degree

Major:

Applications of Multimodal Information (AIM)

End of study internship report

Theme:

Generic Spatiotemporal Scheduling and Navigation for Autonomous UAVs: Reinforcement Learning-based Approaches

Realized by:

Omar Bouhamed

Supervisors:

Dr. Hichem Besbes, Sup'Com, Tunisia
Dr. Hakim Ghazzai, Stevens Institute of Technology, USA
Dr. Yehia Massoud, Stevens Institute of Technology, USA

Work proposed and realized within:

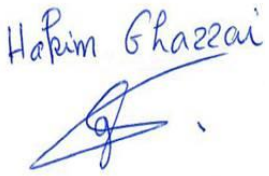
STEVENS INSTITUTE of TECHNOLOGY
THE INNOVATION UNIVERSITY®

Academic year: 2018-2019

Research Supervisor

Dr. Hakim GHAZZAI
Research Scientist at the School of
Systems and Enterprises,
Stevens Institute of Technology, USA

Signature:



Academic Supervisor

Dr. Hichem BESBES
Professor at the Higher School of
Communication of Tunis, Supcom,
University of Carthage, TUNISIA

Signature:

Abstract

In recent years, considerable attention has been given to leverage a variety of smart city applications of unmanned aerial vehicles (UAVs). Characteristics of UAVs including (but not limited to) small size, high mobility in three dimensions, and reliable connectivity, lend their use in several applications, especially ones related to smart cities. Rapid advances in Machine Learning (ML) methods can empower UAVs with autonomous navigation and sensing capabilities, allowing them to adapt to their environments without much human intervention. In this study, we design scheduling and navigation frameworks for autonomous UAVs using Reinforcement Learning (RL).

We first tackle the problem of employing multiple autonomous UAVs to cover the maximum of pre-scheduled events spatially and temporally distributed in a given geographical area over a pre-determined time horizon. The designed framework has the ability to update the planned schedules in case of unexpected emergency events. In this problem, we utilize the Q-Learning algorithm to train the UAVs to learn how to schedule their actions effectively, given the aforementioned problem space. The UAVs use the feedback received from previously taken decisions to learn and adapt its next actions accordingly. A customized reward function is developed to consider several constraints especially the limited battery capacity of the flying units, the time windows of events, and the delays caused by UAV navigation between events.

As for the second part, we investigate the problem of training autonomous UAVs to reach static or mobile destinations in a given three-dimensional ur-

ban area with environmental obstacles that simulate large buildings. In this approach, we develop a deep-learning approach to obstacle-aware UAV navigation. We formulate a continuous action space Deep Deterministic Policy Gradient (DDPG) that is designed to train the UAV to navigate the aforementioned obstacle-laden simulated urban area with the aim of reaching a predefined destination. A customized reward function is developed to minimize the distance separating the UAV and its destination while penalizing collisions.

Numerical simulations investigate the behaviour of the autonomous UAVs for different selected scenarios and corroborate the ability of RL to handle complex vehicle routing problems with several constraints. Additionally, we provide comparisons with optimal deterministic solutions to provide a baseline solution that will aid in validation of the performance of the learning-based solutions we present.

Keywords: Deep learning, reinforcement learning, scheduling, smart city, autonomous navigation, unmanned aerial vehicles, vehicle routing problem.

Dedicated,

To you, my inspiring parents,

To my sister,

To all those I love,

for being the pillows, role models,

cheerleading squad and sounding boards

I have needed.

Acknowledgments

Foremost, I would like to express my sincere gratitude to my advisor *Dr. Hakim Ghazzai*, Research Scientist at Stevens Institute of Technology, for the continuous support, for his patience, motivation, and immense knowledge. His guidance helped me to present a work that I am proud of. I could not have imagined having a better advisor and mentor for my graduation project.

I would like to express my gratitude to *Dr. Yehia Massoud*, Dean of the School of Systems and Enterprises at Stevens Institute of Technology, for welcoming me during my internship and providing me the opportunity of working on such an interesting project.

I would like to offer my special thank to my supervisor *Dr. Hichem Besbes*, Professor at the Higher School of Communication of Tunis, for his precious advice and support.

I thank my fellow lab-mates for the stimulating discussions, for the sleepless nights we were working together, and for all the fun we have had during this internship period.

Last but not the least, my most respectful gratitude is addressed to the members of Jury who agreed to evaluate my work.

Table of contents

Abstract iii

Acknowledgments vi

List of Figures ix

List of Tables xi

List of Acronyms xii

General context 1

0.1 Background & motivation . . . 1
0.2 Related work . . . 2
0.3 Contributions . . . 5
0.4 Report outline . . . 6

1 Introduction to Reinforcement learning 7

1.1 Introduction . . . 7
1.2 RL concept . . . 7
1.3 MDP and RL components . . . 9
1.4 Q-learning . . . 11
1.5 Deep RL . . . 13
1.5.1 Deep QL . . . 14
1.5.2 Deep deterministic policy gradient . . . 15
1.6 Summary . . . 17

2 Investigated use cases for UAVs 18

2.1 Introduction . . . 18
2.2 System model . . . 18
2.2.1 Event model . . . 18
2.2.2 Makespan discretization . . . 20
2.2.3 UAV actions . . . 21
2.2.4 UAV energy model . . . 22
2.3 Scheduling for autonomous UAVs . . . 24

2.3.1 Q-learning inputs 24
2.3.2 Q-Table structure 24
2.3.3 Reward function for scheduling 26
2.4 Emergency procedure and multi-UAV cooperation 29
2.4.1 Emergency procedure 30
2.4.2 Multi-UAVs cooperation 31
2.5 Data gathering for autonomous UAVs 32
2.5.1 Network model 34
2.5.2 Channel model and achievable rate expression 35
2.6 Navigation for autonomous UAVs 38
2.6.1 Urban area model 38
2.6.2 UAV action space 39
2.6.3 Reward function 40
2.6.4 DDPG and transfer learning 41
2.7 Summary 42

3 Simulation and results **43**
3.1 Introduction 43
3.2 Evaluation for UAV scheduling 43
3.3 Evaluation of the multi-UAVs cooperation 49
3.4 Evaluation of UAV data gathering 51
3.5 Evaluation of UAVs Navigation 54
3.6 Summary 58

Conclusion and future work **59**

Accepted/Submitted publications **61**

References **62**

List of figures

1.1 Reinforcement learning: Agent interaction process 7
1.2 Q-learning process steps. 13
1.3 Comparison between QL and deep QL. 14
1.4 Architecture of the actor-critic learning model. 15

2.1 Illustration of the autonomous UAV framework 19
2.2 Illustration of temporal occurrence of the events 20
2.3 An example of a four-level energy division. 23
2.4 Q-table structure. 25
2.5 Metrics illustration 27
2.6 Illustration for the score function 28
2.7 Illustration for the new partition 30
2.8 Illustration of the fleet managment procedure 31
2.9 Illustration of the autonomous UAV data collection 34
2.10 Illustration of the autonomous obstacle-aware UAV navigation . 38
2.11 Example representing a collision scenario 40
2.12 Illustration of the transfer-learning technique. 41

3.1 Temporal and spatial distribution of events 44
3.2 Scenario A: Autonomous scheduling for non-overlapping tasks . 45
3.3 Scenario B: Autonomous scheduling for overlapping tasks. . . . 45
3.4 Convergence speed comparison 46
3.5 Scenario C: Autonomous scheduling with limited battery capacity 47
3.6 Scenario D: Autonomous scheduling with limited battery capacity 47
3.7 Scenario E: Autonomous scheduling with limited battery capacity 48
3.8 Coverage percentage of a sole UAV in function of battery level . 49
3.9 Scenario F: Autonomous scheduling using a fleet of UAVs . . . 50
3.10 Scenario G: UAV data collection path plan 52
3.11 Scenario H: UAV data collection path plan 53
3.12 Completion time of the UAV 53
3.13 Path followed by UAV based on the dynamic target location . . 55
3.14 Obstacle-aware UAV path for environment 1(a) 55
3.15 Obstacle-aware UAV path for environment 2(b) 56

3.16 Obstacle-aware UAV path for environment 2(c) 57
3.17 The reward received by the UAV during the training phase . . . 57

List of Tables

3.1 Simulation parameters for the scheduling framework 44
3.2 UAV activity plan for Scenario F 50
3.3 Simulation parameters for the navigation framework 54
3.4 Task-completion rate 58

LIST OF ACRONYMS

Artificial Intelligence (AI)

Deep Deterministic Policy Gradient (DDPG)

Deep Q-Network (DQN)

Internet Of Things (IoT)

Markov Decision Process (MDP)

Mixed Integer Linear Programming (MILP)

Q-Learning (QL)

Reinforcement Learning (RL)

Traveling Saleman Problem (TSP)

Vehicle Routing Problem (VRP)

General context

0.1 Background & motivation

As the world becomes increasingly interconnected and technology-dependent, a new wave of smart applications is changing how we approach everyday activities. The development of smart cities is going to play a vital role in sustainable development of the world and unmanned aerial vehicles (UAVs), also known as drones, are going to play a major role in it [1]. Nowadays, the efficiency of UAVs is undeniable. UAVs are easy to deploy and have significant flexibility in performing difficult tasks. Additionally, with the use of the uprising Internet-of-things (IoT) technologies, UAVs are involved in a wide range of applications in smart cities, such as capturing live events, monitoring traffic network, delivery of goods [2], disaster management.

Despite the benefits brought from the use of UAVs, several challenges should be addressed to ensure effective operation of the fleet and satisfying applications outputs. Such challenges revolve around the technical specifications of the UAVs and mainly their limited battery capacities [3, 4]. In fact, energy consumption is one of the most crucial challenges confronted by UAVs since they are usually battery powered [5]. Therefore, optimized scheduling and navigation must be designed to effectively determine an activity plan that ensure safe execution of the UAV missions. In general, the activity plan includes to which geographical positions an UAV needs to go and when it needs to recharge its battery given its technical specifications and battery status. In that regard, path

planning is one of key challenges that need to be solved to improve UAV navigation especially in urban areas. The core idea is to devise a collision-free path planning solutions to guide UAVs to reach a given target, while taking into consideration the environment and obstacle constraints in the area of interest.

In addition, in several applications, such as real-time operations and emergency situations, the ability for an UAV to continuously operate without interruption is mandatory. An UAV should be on time to cover a mission with enough energy to accomplish the task. However, a UAV in solo often fails to meet the needs of complicated or long lasting missions. Therefore, a fleet of UAVs is required, thereby the UAVs will be able to substitute their peers in a seamless manner [6]. All these considerations need to be taken into account when designing UAV scheduling solutions.

Traditional scheduling solutions are usually centralized where the UAV operator determines the activity plans of the fleet members. However, the technological trend aspires to attain supporting machines, in our context UAVs, with computational intelligence allowing them to perform in-situation decision making. Hence, the units are capable of making their own decisions while interacting with the surrounding environment without relying on a central unit. In this work, we propose to empower the UAVs with such intelligence in order to self-determine their activity plans and paths to accomplish pre-determined missions, reach their destinations safely, re-schedule their plans to handle emergency situations, and decide to return to their charging stations when needed.

0.2 Related work

A lot of studies have focused on this perplexing problem which is known as vehicle routing problem (VRP). VRPs, considered as one of the most studied combinatorial optimization models, are designed to seek the coverage of a

number of tasks using a fleet of vehicles [7]. These problems consider several constrains and environmental factors in order to yield fast and accurate routes for the agents. VRP is introduced as a generalization of the traveling salesman problem (TSP) presented by Flood [8]. For decades, it has been studied and ameliorated to tackle a variety of problems touching different many engineering domains. For instance, Caceres-Cruz and al. [9] gave a broad overview of different VRP problem combinations and their constraints. The authors presented a variety of routing scenarios that can be found in reality and the most common optimization algorithm developed for addressing all types of rich VRPs (i.e. exact, approximated, and their combinations). In [10], the authors proposed a heuristic algorithm, a modified Clarke and Wright algorithm, to solve a heterogeneous VRP with simultaneous delivery and pick-up. The objective was to design routes that minimize both distance and cost while fulfilling the customer demands. In [11], the authors used the particle swarm optimization (PSO) algorithm to find a set of paths that minimizes the total traveled distance by the agent.

For the particular case of UAVs, some VRP solutions have been proposed. In [12], an approximation algorithm and fast heuristic were developed to solve a generalization of the single VRP with fuel constraints. A mixed-integer linear programming (MILP) formulation was also proposed to find optimal solutions. The objective was to find a path for a single UAV with minimum fuel consumption such that each target is visited at least once by the vehicle without violating the fuel constraint. In [13] and [14], the authors solved MILP problem implementing a generic scheduling framework to manage a fleet of mirco-UAVs to cover spatially and temporally distributed events. An extension of the later work, using low complexity algorithms, is presented in [15]. In [16] and [17], the UAV path planning problems were modeled as mixed integer linear programs (MILP) problem. Also, in [18], a 3D path planning method for multi-

UAVs system or single UAV is proposed to find a safe and collision-free trajectory in an environment containing obstacles. In [19], the authors adopted the ant colony optimization algorithm to determine routes for UAVs while considering obstacle avoidance for modern air defence syste. In [20], a combination of grey wolf optimization and fruit fly optimization algorithms is proposed for the path planning of UAV in oilfield environment. Another scheduling approach of UAVs is presented in [21] where an enhanced version of the ant-colony optimization (ACO) algorithm is utilized for UAVs task allocation and route planning.

However, most of the solutions are based on MILP which are complex or meta-heuristic solutions. The proposed approaches remain centralized where a central node, e.g. a control center runs the algorithm and provides to each flying unit its daily activity plan. This motivates the new research tendency of using artificial intelligence (AI), precisely, reinforcement learning (RL), which allows for the use of a novel trend empowering machines sufficient intelligence to make local decisions and accomplish necessary tasks. The use of autonomous machines that act on people's behalf such as robots, autonomous vehicles, and UAVs is recently increasing at a rapid pace in modern industry [22]. These intelligent machines are expected to play a crucial role in the advancement of smart cities. These smart machines are capable to accomplish more than simply completing tasks. With the power of AI, they learn, evolve, and interact with the environment surrounding them [23, 24]. In [25], a framework for solving a general VRP using RL is proposed. It aims to minimize the total distance traveled in order to cover all tasks. In [26], a quadcopter UAV is used as a waitress to deliver drinks from the bar to customers' tables in a pub using RL. The authors proposed a $\boldsymbol{Q}$-learning (QL) based solution, which uses the evaluative feedback from the customers as reward to adapt the UAV actions accordingly. In [27], the authors presented an implementation of model-based RL method as the high level control of UAV that has to find an efficient route when it is

constrained in battery life. But, these studies did not develop a generic scenario where most important constraints related to smart city applications are handled. Each approach is designed for a specific particular application and hence, some important challenges are ignored when modeling the routing scheduling problem, e.g. the time windows or the occurring of unexpected events. Including time windows and the capability to handle the unexpected events are mandatory for many routing applications such as surveillance, transportation, and delivery system. In these use-cases, each event must be executed within a specific period of time.

0.3 Contributions

This work is motivated by the emergence of various applications demanding the exploitation of autonomous machines, specifically multi-rotor UAVs. The presented work can be divided into two major frameworks. For the first framework, a QL-based approach is designed with the aim to provide activity plans for each autonomous UAV, member of the fleet, to cover the maximum possible of missions while considering several constraints such as the limited battery capacity, the time windows of events, and the travel delays due to the UAV navigation between events. The framework is also designed to be able to update the UAVs pre-scheduled plan in order to cover unexpected events occurring during operating times. An iterative approach is proposed to handle the fleet of UAVs and synchronize their operations. The presented algorithm grants to the UAVs the capability to alternate with their peers smoothly.

While the second framework, unlike existing RL-based solutions which are usually operating on a discretized environment, the proposed framework aims to provide UAV autonomous navigation with continuous action space to reach fixed or moving targets dispersed within a 3D space area while considering the

UAV safety. A deep deterministic gradient decent (DDPG)-based approach is modeled with the objective to allow an UAV determine the best course to accomplish its missions safely, i.e. obstacle avoidance.

0.4 Report outline

This report is divided into three chapters. In Chapter 1, we introduce RL concept and some selected approaches. Afterwards, in Chapter 2, we investigate use cases related to autonomous UAVs in smart city applications using the reinforcement learning technique, while in the last Chapter, we study the behavior of the autonomous UAVs for various selected scenarios and environments.

Chapter 1

Introduction to Reinforcement learning

1.1 Introduction

In this chapter, we introduce the concept of RL. We also define selected approaches and algorithms to deal with different situations.

1.2 RL concept

Machine learning is an application of artificial intelligence (AI) that provides systems the ability to automatically learn and improve from experience without



Figure 1.1: Reinforcement learning: Agent interaction process within the environment.

being explicitly programmed. Machine learning focuses on the development of computer programs that can access data and use it to learn for themselves. There are three classes of machine learning: unsupervised learning, supervised learning, and RL [28]. Each of these is good at solving a specific set of problems. Unsupervised learning tries to find hidden structure/ pattern in unlabeled data. It is used for clustering, dimensional reduction, feature learning, and density estimation, among other tasks. Supervised learning, unlike the unsupervised learning, works on a complete labeled data set. This category of learning is good at creating classification models for discrete data and regression models for continuous data. The machine learning or neural network model produced by supervised learning is usually used for prediction.

Finally, RL is learning how to map situations to actions so as to maximize a numerical reward signal. The learner is not told which actions to take, but instead it must discover which actions yield the most reward by learning and testing during a training phase. In the most interesting and challenging cases, actions may affect not only the immediate reward but also the next situation and, through that, all subsequent rewards. These two characteristics trial-and-error search and delayed reward are the two most important distinguishing features of RL. Simply put, the idea behind RL is that it enables an agent to learn in an interactive environment by trial and error using feedback from its own actions and experience as illustrated in Fig. 1.1. The reinforcement learning system is defined by four main sub-elements:

- Policy : the policy is the agent's strategy. It is a map of action selection. It means it gives the probability of taking particular action in a particular state.

- Reward: for each action taken by the agent, the environment respond with a reward signal, as a feedback.

- Value function: the value function indicates how worth to be in such state. It is estimated from the observations that the agent makes from his environment.

- Model of the environment: It imitates the behaviour of the environment. Given the state and action as input of the model, it can predict the next state and next action, which means deciding on action by considering possible future situations before they are actually experienced.

1.3 MDP and RL components

As previously stated, RL is a type of Machine Learning. It allows machines and software agents to automatically determine the ideal behavior within a specific context, in order to maximize its performance. Simple reward feedback is required for the agent to learn how to behave, this is known as the reinforcement signal. There are many different algorithms that tackle this issue. As a matter of fact, RL is about decision making based on last observation. The interaction between the agent and the environment is modeled as a Markov Decision Process (MDP), where basic reinforcement is modeled as:

- a set of states, S,

- a set of actions, A,

- Pr is the probability of transition from state s to state s' under action a,

- R the reward function,

where the states are markovien:

$$Pr[S_{t+1}|S_t] = Pr_s[S_{t+1}|S_t, S_{t-1}, S_{t-2}, \ldots, S_0] \tag{1.1}$$

Based on the current state, the agent executes an action, receives a reward evaluating its behavior and it transits to a new state. In order to have the best behavior, RL need to maximize the expected cumulative reward. The cumulative reward, at each time step t, can be written as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \tag{1.2}$$

where γ is the discount factor essentially determines how much the reinforcement learning agents cares about rewards in the distant future relative to those in the immediate future. If ($\gamma = 0$), the agent will be short-sighted and only learn about actions that produce an immediate reward, while if ($\gamma = 1$), that means the agent is far-sighted, i.e. it cares about all future rewards.

The value function, denoted as $v(s) = \mathbb{E}_\pi[G_t|S_t = s]$, is introduced to measure how good a state is to the agent. Among all possible value-functions, there exist an optimal value function that has higher value than other functions for all states, defined as:

$$v^*(s) = \max_\pi v_\pi(s) \forall s \in S \tag{1.3}$$

In addition to the value-function, RL algorithms introduce another function which is the state-action pair, defined as $Q(s, a)$. Q is a function of a state-action pair and returns a real value. Since $v^*(s)$ is the maximum expected total reward when starting from state s, it will be the maximum of optimal Q-function $Q^*(s, a)$ over all possible actions. Therefore, the relationship between $Q^*(s, a)$ and $v^*(s)$ is obtained as:

$$v^*(s) = \max_a Q^*(s, a) \forall s \in S, \tag{1.4}$$

and as long as the optimal $Q^*(s, a)$ is known, the optimal policy is extracted

by choosing the action a that gives maximum $Q^*(s,a)$ for state s. The optimal policy is presented as:

$$\pi^*(s) = \arg\max_a Q^*(s,a) \forall s \in S, \tag{1.5}$$

To solve MDPs, we have two approaches: value and policy iteration. Value iteration computes the optimal value function by improving the estimate $v(s)$. Policy iteration is the combination of policy evaluation and policy improvement, where policy evaluation computes the value function corresponding to the the policy, while the policy improvement consists of improving this value function. These approaches are based on Bellman's equation [29], which allow us to represent the $v(s)$ as a recursive relationship between the value of a state and the value of its successor states.

$$\begin{aligned} v(s) &= \mathbb{E}[G_t | S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) | S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma \max_a Q^*(s_{t+1}, a') | S_t = s] \end{aligned} \tag{1.6}$$

1.4 Q-learning

Tabular methods refer to problems in which the state and actions spaces are small enough for approximate value functions to be represented as arrays and tables. In this case, we can apply simple tabular Q-learning (QL), QL is a model-free reinforcement learning algorithm. The goal of QL is to learn a policy, which tells an agent what action to take under what circumstances, state s. It does not require a model, hence the connotation "model-free" of the environment, and it can handle problems with stochastic transitions and rewards, without requiring adaptations. This approach consists in keeping a table of the values of Q(s,a)

for all states s and actions a, defined as Q-table. This table is initialized to a constant values, then, traces (experiences) are collected by some exploration method. At each time step, we apply the following update inspired from the Bellman equation to our Q-table:

$$Q(s,a)^{new} \leftarrow (1-\alpha)Q(s,a)^{old} + \alpha\left(R(s,s_{t+1}) + \gamma \max_a Q(s_{t+1},a_{t+1})\right), \tag{1.7}$$

where α is the learning rate. This iterative improvement, is known as Time-Difference (TD) Learning (refers to the fact of looking ahead only one step, then compute the current state value), updates Q(s,a) by slightly moving it towards a more complete approximation using the current reward and observed next state. The agent starts to interact with the environment, and upon each interaction the agent will observe the reward of its action $R(s,s_{t+1})$ and the state transition (new state s_{t+1}). Then use the above equation to update its own estimate of Q(s,a) as illustrated in Fig. 1.2.

Exploration vs exploitation

In order to avoid falling in a local maximum, in order to efficiently manage the exploration/exploitation trade-off, an exploration policy is employed.

- Exploration is finding more information about the environment.
- Exploitation is exploiting known information to maximize the reward.

The most studied exploration policy is ϵ-greedy exploration, which consists in taking a random action with probability ϵ (explore) and the currently estimated best action $\arg\max_a Q(s,a)$ with probability $1-\epsilon$ as shown in flowchart 1.2. The ϵ-value can be decreased overtime as the agent becomes more confident with its estimate of Q-values.

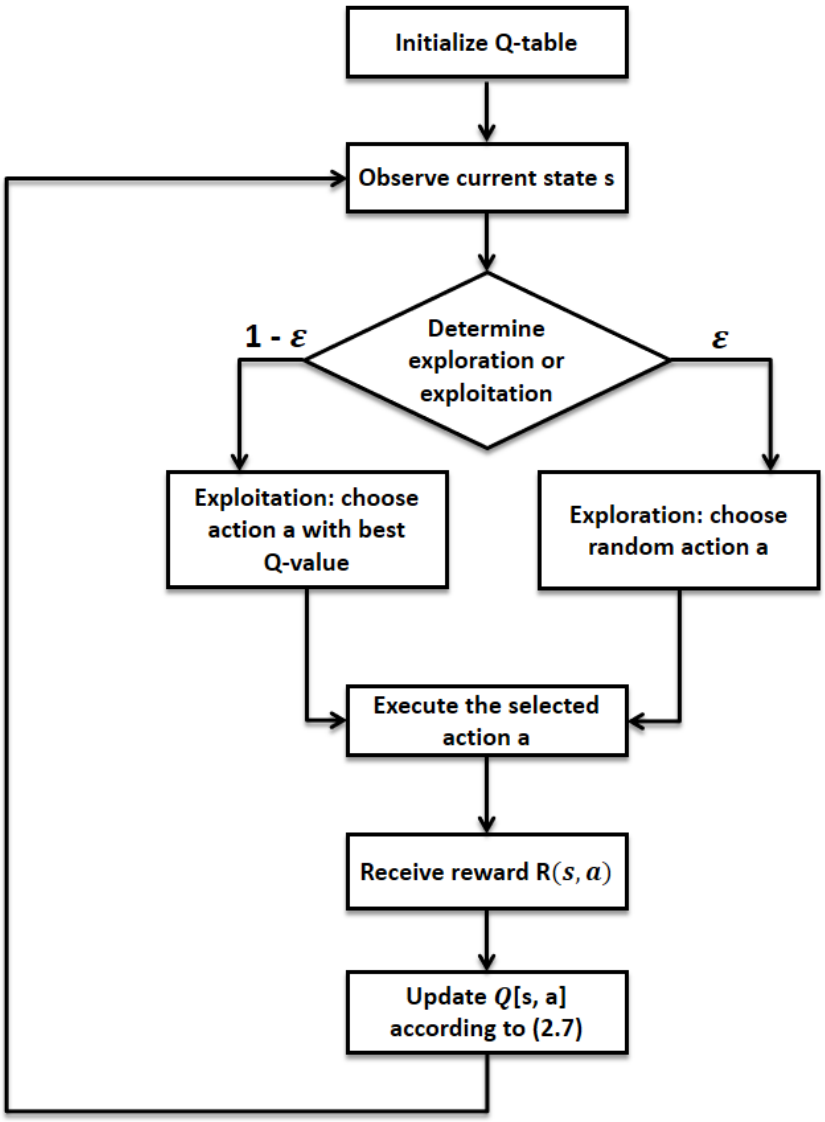


Figure 1.2: Q-learning process steps.

1.5 Deep RL

In the case of large (infinite) state space or action space, tabular approaches becomes impossible to use. In such cases, substitute tabular methods are substituted for neural networks-based approaches. Moreover, in this case, an exact optimal policy cannot be found, instead a good approximate solution is sought. Problems with approximate solutions are essentially memory problem and time. The major advantage of function approximation is generalisation: generalize from seen to unseen state similar to the current one. In other words, it takes examples from value function and tries to generalize and construct an approxi-

mation of the entire function.

In the next part, we will focus on methods with Neural Network function approximator to deal with deep RL, thanks to their successful results in the literature even we do not have guarantees of convergence.

1.5.1 Deep QL

Deep QL is a technique introduced in [30], which combines QL, which is a classical form of reinforcement with deep neural networks. As previously mentioned, in deep Rl, we use a neural network to approximate the value function instead of the tabular form used in simple tabular approaches. For the case of Deep QL, the state is given as the input and the Q-value of all possible actions is generated as the output. The comparison between QL and deep QL is illustrated below:

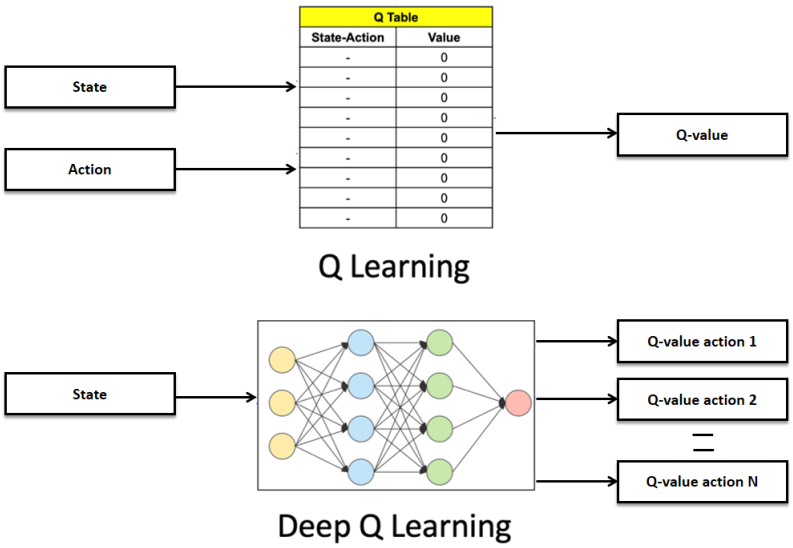


Figure 1.3: Comparison between QL and deep QL.

We define the $Q(s, a)$ function as a neural network Q_θ defined by a set of parameters θ that maps (s, a) to the corresponding Q-value. The $Q(s, a)$ neural network can be trained by stochastic online gradient descent. At time step t, in case we wish to update the parameters θ_{t-1} with the experience tuple (s_t, a_t, s_{t+1})

that was just collected, we use the following loss function to optimize θ_t:

$$\mathcal{L}_t(\theta) = \left(Q(s_t, a_t|\theta) - y_t\right)^2, \tag{1.8}$$

$$y_t = R(s_t, s_{t+1}) + \gamma \max_a Q(s_{t+1}, a|\theta_t) \tag{1.9}$$

1.5.2 Deep deterministic policy gradient

Policy gradient methods are fundamental in reinforcement learning techniques that rely on optimizing parameterized policies with respect to cumulative reward by gradient descent. In the previous part, we focused on value-based methods: to choose action given a state, the action with the highest Q-value is chosen. However, policy gradients methods do not require the computation of value function since they approximate directly the optimal policy. Instead, value function is used, only,to learn policy parameters.

DDPG was developed as an extension of deep Q-network (DQN) algorithms introduced by Mnih et al. [31], which was the first approach combining deep and reinforcement learning but only by handling low-dimensional action spaces. DDPG is also a deep RL algorithm, that has the capability to deal with large-dimensional/infinite action spaces. It tries to find an efficient behavior strategy for the agent to obtain maximal rewards in order to accomplish its assigned

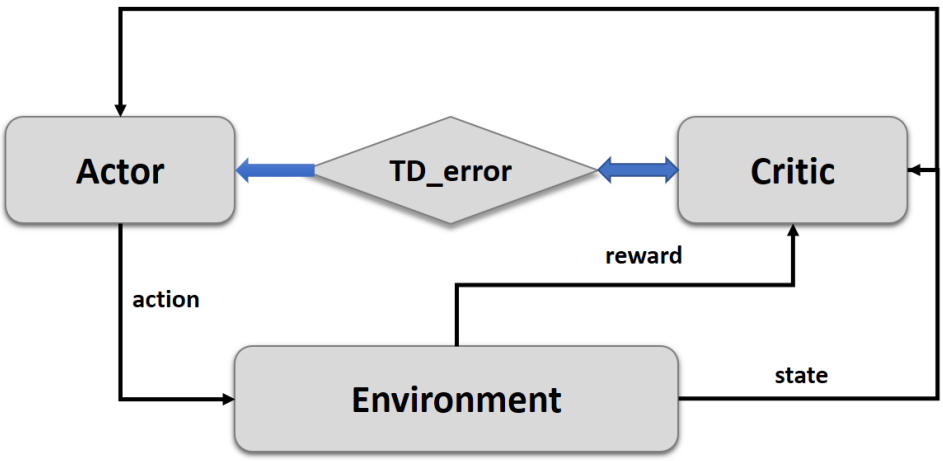


Figure 1.4: Architecture of the actor-critic learning model.

tasks [32]. This DPG algorithm has the capability to operate over continuous action spaces which is a major hurdle for classic RL methods like Q-learning.

DDPG is based on the actor-critic algorithm. It is essentially a hybrid method that combines the policy gradient and the value function together. The policy function π is known as the actor, while the value function Q is referred to as the critic. Essentially, the actor output is an action chosen from a continuous action space, given the current state of the environment $a = \pi(s|\theta^{\pi})$. As for the critic, its output $Q(s, a|\theta^{\pi})$ is a signal having form of a Temporal Difference (TD) error to criticize the actions made by the actor knowing the current state of the environment. A diagram summarizing the actor-critic architecture is given in Fig. 1.4.

Note that the training phase of the DDPG model is executed for M episodes where each one of them accounts for T steps. We use the index t to denote an iteration within a single episode where $t = 1, \ldots, T$. The actor and critic are designed with neural networks. The value network is updated based on Bellman equation [29] by minimizing the mean-squared loss between the updated Q value and the origin value, which can be formulated as shown in Algorithm 1 (line 11). As for the policy network's update, it is based on the deterministic policy gradient theorem [33] Algorithm 1 (line 13).

There are also some practical tricks that are used to enhance the performance of the framework. A trade off between exploration and exploitation is made by the use of ϵ-greedy algorithm, where a random action a_t is selecting with ϵ probability, otherwise a precise action $a_t = \pi(s_t|\theta^{\pi})$ is selected according to the current policy with a $1 - \epsilon$ probability. Furthermore, an experience replay buffer b, with size B, is used during the training phase to break the temporal correlations. Each interaction with the environment is stored as tuples in the form of $[s_t, a, r, s_{t+1}]$, which are the current state, the action to take, the reward of performing action a at state s_t, and the next state, respectively (Algorithm 1

(line 9)) and, during the learning phase, a randomly extracted set of data from the buffer is used (Algorithm 1 (line 10)). Also, target networks are exploited to avoid the divergence of the learning algorithm caused by the direct updates of the networks weights with the gradients obtained from the TD error signal.

Algorithm 1 DDPG

1: Randomly initialize critic $Q(s, a|\theta^{\pi})$ and actor $\pi(s|\theta^{\pi})$ neural networks with weights θ^{Q} and θ^{π}.
2: Initialize target networks Q' and π' with weights $\theta^{Q'} \leftarrow \theta^{Q}$, $\theta^{\pi'} \leftarrow \theta^{\pi}$.
3: Initialize replay buffer b.
4: **for** episode = 1,...,M **do**
5: Receive first observation s_1.
6: **for** t = 1,...,T **do**
7: Select a_t based on ϵ-greedy algorithm: select random action a_t with ϵ probability, otherwise $a_t = \pi(s_t|\theta^{\pi})$ according to the current policy.
8: Execute action a_t and observe reward r_t and new state s_{t+1}.
9: Store transition $[s_t, a_t, r_t, s_{t+1}]$ in b.
10: Sample a random batch of N transitions $[s_j, a_j, r_j, s_{j+1}]$.
11: Set $y_j = r_j + \gamma Q'(s_{j+1}, \pi'(s_{j+1}|\theta^{\pi'})|\theta^{Q'})$.
12: Update critic by minimizing the loss:
$L = \frac{1}{N}\sum_j (y_j - Q(s_j, a_j|\theta^{Q}))^2$
13: Update the actor policy using policy gradient:
$\nabla_{\theta^{\pi}}\pi|_{s_j} \approx \frac{1}{N}\sum_j \nabla_a Q(s, a|\theta^{Q})|_{s=s_j, a=\pi(s_j)} \nabla_{\theta^{\pi}}\pi(s|\theta^{\pi})|_{s_j}$
14: Update the target networks:
$\theta^{Q'} \leftarrow \nu\theta^{Q} + (1-\nu)\theta^{Q'}$
$\theta^{\pi'} \leftarrow \nu\theta^{\pi} + (1-\nu)\theta^{\pi'}$
15: **end for**
16: **end for**

1.6 Summary

In this chapter, we presented reinforcement learning algorithms from tabular to approximate function solutions. In the next chapter, we present our use cases and describe our studied model before discussing results.

Chapter 2

Investigated use cases for UAVs

2.1 Introduction

In this chapter, we investigate use cases related to autonomous UAVs in smart city applications using the reinforcement learning techniques.

2.2 System model

In this study, we consider a geographical area of size $X \times Y \times Z$ km^3 where a fleet of UAVs is employed to execute different tasks during a time span Γ. We assume that the flying units have the capability to wirelessly exchange messages and communicate with a control center, for example to get updates regarding emergencies and unexpected events. We refer to each UAV as agent d_i, $i = 1, \ldots, D$, where D denotes the fleet size.

2.2.1 Event model

The environment surrounding the UAVs is a dynamic and, in this study, we aim to provide a framework that gives the UAVs more autonomy and intelligence to be able to deal with the dynamism of the surroundings. Therefore, we consider two types of events expected, e.g., the UAV can be equipped with

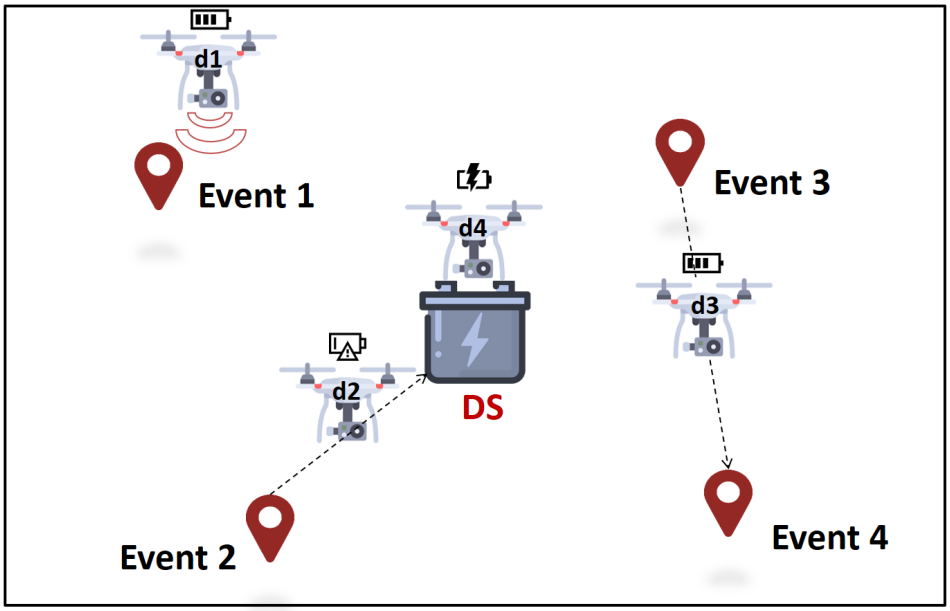


Figure 2.1: Illustration of the autonomous UAV framework. In this example, four UAVs are employed to cover four events. The $\mathcal{DS}$ is made available to allow UAVs recharge their batteries when needed.

specific sensors to collect data from different locations as a daily task, and unexpected, e.g., in case of accident, the UAV fly to an accident's location to get a detailed report about the accident and send the report in real-time to the related authorities. Initially, the UAVs intent to cover the maximum number of E pre-scheduled events. They have also the capability to adjust trajectory (midway) in order to cover unexpected events. The E events take place in predefined locations during a makespan Γ. Each event e, where $e = 1, \ldots, E$, is defined by the parameters st_e, pt_e, loc_e and pr_e, which are the event starting time, the event processing time, the $3D$ geographical location $loc_e = (x_e, y_e, z_e)$, and the event priority, respectively. The set of the pre-defined events is denoted by $\mathcal{E}$. We identify the docking station as the event $e = 0$, denoted by $\mathcal{DS}$ and located at $loc_0 = (x_0, y_0, z_0)$, which serves as a charging station.

Regarding the unexpected events, each one of them, denoted by the index u, possesses the same characteristics as the predefined events, expect the fact that their parameters are unknown until they occur i.e., the UAV is alerted of the existence of an emergency event along with its specs (location, duration

and priority level) once it appears, thus the UAV adjusts its path if required, accordingly. The set of the unexpected events is denoted by $\mathcal{U}$. The starting time of an unexpected event u, denoted as st_u, is the moment when the UAV is notified by the event, pt_u is the duration for the unexpected events, which can be either a fixed period, e.g., 10 minutes during which the UAV needs to attend the event or flexible period depending on the nature of the reported event. Finally, pr_u denotes the event priority, where pr_u is greater or equal than the priority of an predefined event e ($pr_e \leq pr_u$) to encourage the UAVs to cover the emergent events first.

2.2.2 Makespan discretization

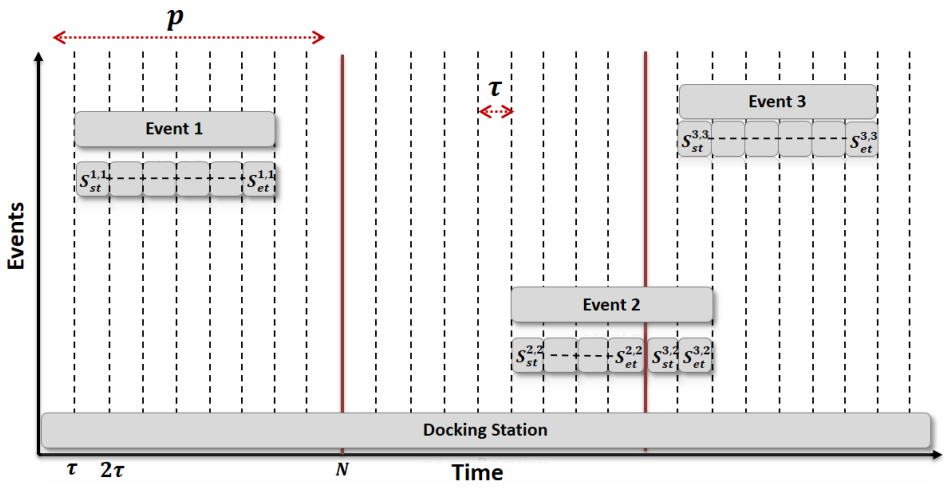


Figure 2.2: Illustration of the time characteristics and temporal occurrence of the events in a discretized makespan. The figure shows $E = 3$ events situated in a $P = 3$ periods. The $\mathcal{DS}$ is assumed to be available all along the makespan so that the UAVs can return to it during periods of inactivity or to charge their batteries.

In this study, with the aim of dealing with the time dimension challenge and to handle efficiently the unexpected events, we divide the makespan Γ into P coextensive time periods referred as p, where $p = 1, \ldots, P$. These periods reflect the limited knowledge of the UAVs about the future. Hence, the UAV can identify only the events occurring within the next time period. Likewise,

each time period p is divided into N sub-periods $(s_1^p, \ldots, s_N^p)$ having the same length τ as depicted in Fig. 2.2. The value of τ plays a key role in controlling the complexity and the rapidity of the proposed solution, i.e., increasing τ leads to a faster and lower complexity solution but may lead to unsatisfactory performance and vice versa. In every period P, we associate to each event e a certain number of time periods corresponding to the slots during which the event takes place. We define the set of time periods $\mathcal{T}_{p,e}$ associated to event e during period p as follows: $\mathcal{T}_{p,e} = \{s_{st}^{p,e}, \ldots, s_{et}^{p,e}\}$ where $s_{st}^{p,e}$ denotes the time period where the event e starts while $s_{et}^{p,e}$ corresponds to the time period at which the event e takes end in period p. In the case where the event occurs during two different time periods, the later is divided into two sub-events as shown in Fig. 2.2.

2.2.3 UAV actions

Each UAV is characterized by its average speed V_d, its current battery state b_d, its battery capacity $\bar{b}_d$ ($0 \leq b_d \leq \bar{b}_d$), and the battery charging rate c_d measured in Watt. Initially, we consider that the UAVs are placed at the docking station $\mathcal{D}S$. Without loss of generality, we assume that its starting time and ending time are zero and Γ, respectively. In general, the UAV has four possible modes:

• waiting at $\mathcal{D}S$: this mode corresponds to a 'Standby' status during periods of inactivity. We assume that during the absence of events, the UAV will need to return to the $\mathcal{D}S$.

• executing an event e: the UAV should be located at loc_e to be able to cover the event.

• moving from a position e to another e': the UAV will move towards an event to cover it or return to the docking station $\mathcal{D}S$. We define $fl_{e,e'}^d$ as the flying time

spent by the UAV to reach event e' from event e and it is expressed as follows:

$$fl^{d}_{e,e'} = \frac{dist(loc_e, loc_{e'})}{V_{\mathrm{d}}}, \tag{2.1}$$

where $dist(loc_e, loc_{e'})$ the $3D$ euclidean distance between the two events' locations.

• charging at $\mathcal{D}S$: this mode corresponds to the state when the UAV is charging its battery whenever it is lower than $\bar{b}$ with the charging rate c_d.

2.2.4 UAV energy model

In our framework, we consider that the power consumption of the UAV is mainly composed of two components, one related to the propulsion and the other to the mounted devices, e.g., communication interface and sensors. In this study, we adopt the UAV power consumption due to propulsion component recently presented in [34]. It is expressed as follows:

$$\begin{aligned} J(V_d) =& J^{bl}\left(1 + \frac{3{V_d}^2}{U_{tip}^2}\right) + J^{ind}\left(\sqrt{1 + \frac{{V_d}^4}{4v_0^4}} - \frac{{V_d}^2}{2v_0^2}\right)^{\frac{1}{2}} \\ &+ \frac{1}{2} d_0 \rho s A {V_d}^3, \end{aligned} \tag{2.2}$$

where J^{bl} and J^{ind} are two constants representing the blade profile power and induced power in hovering status, respectively, U_{tip} denotes the tip speed of the rotor blade, v_0 is known as the mean rotor induced velocity in hover, d_0 and s are the fuselage drag ratio and rotor solidity, respectively, ρ and A denote the air density and rotor disc area, respectively. In order to obtain to the power consumption for hovering, we simply substitute V_d in (2.2) by 0 and then $J^{hov} = J(0) = J^{bl} + J^{ind}$. As for the required power to execute an event, denoted by J^{ex}, we assume that the UAV executes its tasks while hovering statically. Hence,

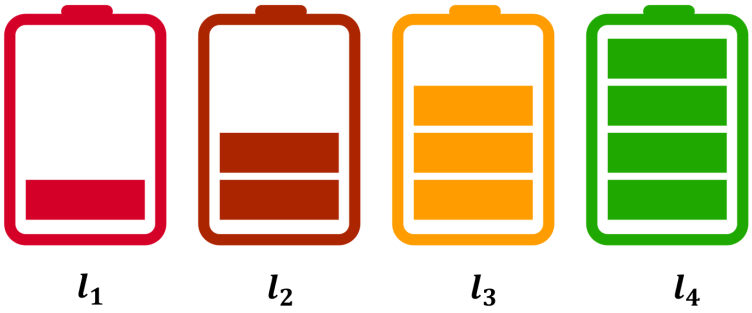


Figure 2.3: An example of a four-level energy division.

It is expressed as $J^{ex} = J^{hov} + J^{md}$, where J^{md} is the mounted devices related power during reception.

The battery level of the UAV, denoted by b_d, can vary according to its three possible modes described earlier:

• 'Standby' mode: the UAV battery is charged as follows:

$$b_d(s_{n+1}) = \min(\bar{b}_d, b_d(s_n) + c_d \cdot \tau), \tag{2.3}$$

where $b_d(s_n)$ is the battery level at time period s_n.

• covering event e: The battery level is reduced as follows:

$$b_d(s_{n+1}) = \max(0, b_d(s_n) - J^{rx} \cdot \tau). \tag{2.4}$$

• moving from a position to another: In this case, the UAV battery is updated as follows:

$$b_d(s_{n+1}) = \max(0, b_d(s_n) - J(V_d) \cdot \tau). \tag{2.5}$$

In this study, we propose to divide the battery capacity into K energy levels. Thus, each UAV has K battery charging states that we refer as $\mathcal{L} = \{l_1, \ldots, l_K\}$, where the highest energy level $l_K = [\bar{b}(1 - \frac{1}{K}), \bar{b}]$ and the lowest level $l_1 = [0, \frac{\bar{b}}{K}]$. An illustration of a $K = 4$ energy levels is shown in Fig. 2.3.

2.3 Scheduling for autonomous UAVs

In this part, we propose to use one of the fastest and simplest-structured RL algorithms: the QL, introduced in Section 2.4. In this section, we will describe the different elements of the employed approach and the adapted methods.

2.3.1 Q-learning inputs

In order to use QL, we associate to the autonomous UAV a reward function $\mathcal{R}$ and the sets of states and actions during the investigated period, denoted by $\mathcal{S}_p$ and $\mathcal{A}_p$, respectively. To define the latter sets, we introduce the set of missions $\mathcal{M}_p$, where $\mathcal{M}_p$ is the set of pre-scheduled and unexpected events during a period p, identified as follows: $\mathcal{M}_p = \mathcal{E}_p \cup \mathcal{U}_p$. Hence, $\mathcal{M}_p$ is defined as $\mathcal{M}_p = \{m, \text{where } m = 0, \ldots, M\}$, where M is the total number of missions. In addition, we define the set of time periods associated to a mission m during a period p, denoted by $\hat{\mathcal{T}}_{p,m}$, as: $\hat{\mathcal{T}}_{p,m} = \left\{s_1^{p,m}, \ldots, s_{st-1}^{p,m}, s_{st}^{p,m}, \ldots, s_{et}^{p,m}, s_{et+1}^{p,m}, \ldots, s_N^{p,m}\right\}$. The set $\hat{\mathcal{T}}_{p,m}$ includes all the time periods of the makespan to which we assign the index of the mission m. Since the UAV can be located at any mission during any time period with any battery charging state $\in \mathcal{L}$, we create the set of states $\mathcal{S}$ by concatenating the different $\hat{\mathcal{T}}_{p,m}, \forall m = 0, \ldots, M$, while considering the possible battery level states defined in Section II, as follows: $\mathcal{S} = \left\{\hat{\mathcal{T}}_{p,0}^{l_1}, \ldots, \hat{\mathcal{T}}_{p,M}^{l_1}, \ldots, \hat{\mathcal{T}}_{p,0}^{l_n}, \ldots, \hat{\mathcal{T}}_{p,M}^{l_n}\right\}$. As for the set of actions, it is defined as $\mathcal{A} = \mathcal{M}$ that corresponds to the set of missions. Hence, at each action, the UAV will choose one of the mission to go to given its current state.

2.3.2 Q-Table structure

QL revolves around the notion of updating the elements of a table $\boldsymbol{Q}$ that we present its structure in Fig. 2.4. The elements of the Q-table denote the values of doing an action $a \in \mathcal{A}$ when the UAV is at a state $s \in \mathcal{S}$. Initially, the $\boldsymbol{Q}$-table

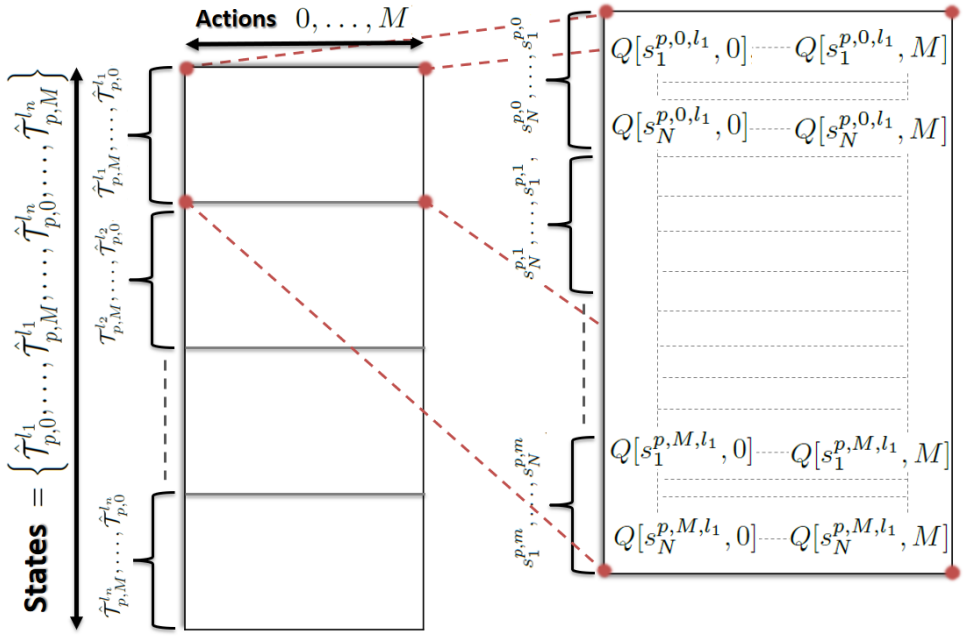


Figure 2.4: Q-table structure.

is filled with zero-elements. Note that the training phase of the QL algorithm is executed for K_{ep} episodes where each one of them accounts for I iterations. The update rule, discussed in Section 2.4 (1.7), is used to update the $\boldsymbol{Q}$-table of the learning algorithm:

$$\begin{aligned} Q[s_n^{p,m,l}, a](i) =& (1-\alpha) Q[s_n^{p,m,l}, a](i-1) \\ &+ \alpha[\mathcal{R}(s_n^{p,m,l}, a) + \gamma \max_{a \in \mathcal{A}} Q[s_{n'}^{p,m',l'}, a](i+1)], \end{aligned} \tag{2.6}$$

where i denotes the current iteration of the QL algorithm where $i = 1, \ldots, I$. The parameter $\mathcal{R}(s_n^{p,m,l}, a)$ measures the reward corresponding to the decision of taking action a when the agent is at the state $s_n^{p,m,l}$. Regarding the last term in (2.6), $\max_{a \in \mathcal{A}} Q[s_{n'}^{p,m',l'}, a](i+1)$, it returns the highest $\boldsymbol{Q}$ value among all possible actions in $\mathcal{A}$, related to the new state $s_{n'}^{p,m',l'}$.

At each iteration, the UAV is assumed to progress in time according to an exploration or exploitation decision depending on the value of ε. In exploration, an action is chosen randomly, while, in exploitation, the action is chosen such as the corresponding element in $\boldsymbol{Q}$ is maximized as addressed in chapter 2. At each

progress step, one element of $\boldsymbol{Q}$ is updated as given in (2.6). At the beginning of an episode, the time is reset and the UAV is returned to $\mathcal{D}S$.

To keep balance between exploration and exploitation actions, an ε-greedy action-selection policy is employed. In this work, the following update rule for ε value is adopted with the motivation of acquiring a wise exploration-exploitation balance ensured using the following expression:

$$\varepsilon_{\text{new}}(i) = \varepsilon_{\text{min}} + (\varepsilon_{\text{max}} - \varepsilon_{\text{min}}) \exp(-dr \cdot i), \tag{2.7}$$

where dr is the decay rate, i refers to the iteration of the QL algorithm, ε_{min} and ε_{max} refers to the minimum and maximum exploration probability, respectively.

2.3.3 Reward function for scheduling

Recall that, in our setting, we aim to maximize the total coverage of the tasks while taking into account the limited battery capacity, the delay needed to move from a task to another, the priority of the missions and the reaction of the unexpected events. If the UAV has no mission to cover, it is encouraged to return to the $\mathcal{DS}$ to either charge its battery in case of low battery charging state or to go into standby mode. To deal with the flying time constraint, the reward function of the UAV is expressed as a function of the starting and ending times ($s_{st}^{p,m,l}$ and $s_{et}^{p,m,l}$) of the missions as well as the flying time spent between them $fl_{m,m'}^{d}$.

In the following, we define three time metrics employed to evaluate the benefit of the chosen action:

• The arrival time of the UAV at mission m chosen after being at state $s_n^{p,m,l}$, denoted by $AT(s_n^{p,m,l}, a)$, is expressed as:

$$AT(s_n^{p,m,l}, a) = s_n \cdot \tau + fl_{m,a}^{d}, \tag{2.8}$$

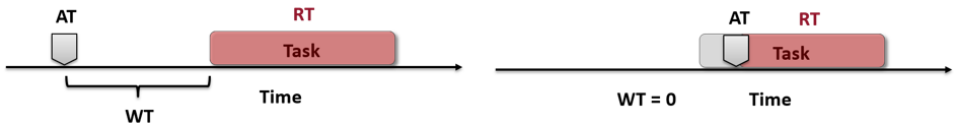


Figure 2.5: Illustration of the metrics changes as a function of the arrival time AT.

• The waiting time that the UAV must wait before the start of mission m after leaving the state $s_n^{p,m,l}$, denoted by $WT(s_n^{p,m,l}, a)$, is expressed as:

$$WT(s_n^{p,m,l}, a) = \begin{cases} s_{st}^{m,a}\,\tau - AT(s_n^{p,m,l}, a), & \text{if } s_{st}^{p,m} \leq AT(s_n^{p,m}, a), \\ 0, & \text{otherwise.} \end{cases} \tag{2.9}$$

• The remaining time left in mission a if the agent arrives at a after being at state $s_n^{p,m,l}$, denoted by $RT(s_n^{p,m,l}, a)$, is expressed as:

$$RT(s_n^{p,m,l}, a) = \begin{cases} s_{et}^{p,a}\,\tau - AT(s_n^{p,m,l}, a), & \text{if } s_{st}^{p,m} \leq AT(s_n^{p,m}, a), \\ (s_{et}^{p,a} - s_{st}^{p,a})\,\tau, & \text{otherwise.} \end{cases} \tag{2.10}$$

The metrics AT, WT and RT are illustrated in Fig. 2.5. The metric AT is added to encourage the UAV collect the data from the sensor when the data acquisition time window is active. The UAV is penalized if it arrives to sensor node too early, i.e., WT is high. The remaining time left metric RT is considered to encourage the UAV go to sensors as soon as possible once the data to be collected is available. This helps in accelerating the data collection process from all the nodes. The reward function, $R(s_n^{p,m,l}, a)$, measures the worthiness of covering a mission m when the UAV is located at state $s_n^{p,m,l}$ and is formulated as follows:

$$\mathcal{R}(s_n^{p,m,l}, a) = \begin{cases} \Psi(s_n^{p,m,l}, a) - \Psi(s_n^{p,m,l}, m), & \text{if } a \neq m \\ \Psi(s_n^{p,m,l}, m), & \text{otherwise,} \end{cases} \tag{2.11}$$

where $\Psi(s_n^{p,m,l}, a)$ is a score function having values $\in (0, 100 \cdot \bar{pr})$, where

$\bar{pr}$ is the highest priority that can be assigned to a mission so it is executed first. The score Ψ measures the worth of mission m when the UAV is located at state $s_n^{p,m,l}$. Fig. 2.6 depicts an example of the score function Ψ with respect to the time window of the task and discusses different cases. The score function is built to encourage the UAVs to cover tasks in case of filled battery (events score higher than $\mathcal{DS}$ score) and vice versa, in case of empty battery.

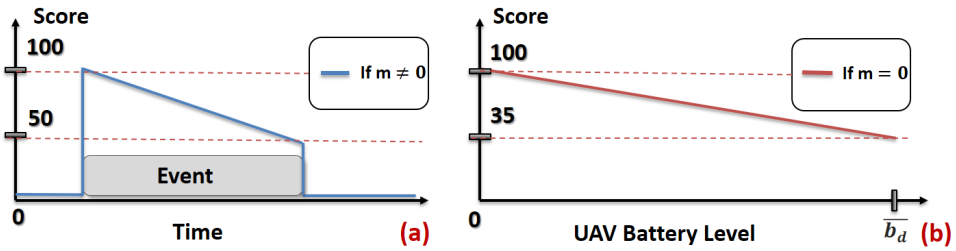


Figure 2.6: Score function Ψ: (a) illustration of the score of the missions except $\mathcal{DS}$. Initially, it is equal to 0 till it reaches its maximum value when $RT =$ task length and them keeps decreasing till the end of the mission. As for the $\mathcal{DS}$ (b), the score is as function of the UAV battery level. The score is a linear function that reaches its maximum 100 when $b_d = 0$ and its minimum at $b_d = \bar{b}_d$.

In this study, to make the UAV prioritizes the mission with higher priority, the score function for missions except $\mathcal{DS}$ is multiplied by pr_m. Additionally, with the intention of better management and efficient exploitation of the UAV's energy, a score function depending on the UAV battery charging state, is assigned to the $\mathcal{DS}$, the origin. Furthermore, an energy mask is added as a safety margin for the UAV to avoid crashes, thus if b_d goes lower than the security bound $sc_{d,p}$ the UAV is encouraged to go back to the $\mathcal{DS}$, where $sc_{d,p}$ is a dynamic variable specific to each time period p calculated as the maximum energy required for the UAV d to travel from the farthest mission in period p to reach the $\mathcal{DS}$. Hence, the expression of Ψ is given as follows:

$$\Psi(s_n^{p,m,l},a)=\begin{cases} pr_m \max\left(50\left(\frac{RT(s_n^{p,m},a)}{(s_{et}^{p,a}-s_{st}^{p,a})\cdot\tau}+1\right)\cdot\frac{1}{1+WT(s_n^{p,m,l},a)},0\right), \\ \qquad\qquad\qquad\qquad \text{if } a\in\mathcal{A}\backslash\{0\}, \\ 100\left(-0.65\cdot\frac{b_d}{b_d}+1\right), \quad \text{if } a\in\{0\}, \end{cases}$$

$$\textbf{if } sc_{d,p}\leq b_d,$$

$$\Psi(s_n^{p,m,l},a)=\begin{cases} 0, & \text{if } a\in\mathcal{A}\backslash\{0\}, \\ 100, & \text{if } a\in\{0\}, \end{cases} \quad \textbf{otherwise.} \tag{2.12}$$

In the reward function, if $a \neq m$, i.e., the UAV is moving toward a different mission than the the current one, we subtract $\Psi(s_n^{p,m,l}, m)$, which is the score of deciding to stay in the same state for the next time period, from the score to move to a new state. This subtraction encourages the UAV to remain at the same mission in case RT of the current mission is still significant.

During the offline training phase, the UAV will learn to head towards the events, including the $\mathcal{DS}$, maximizing its reward function. The UAV will be able to cover the pre-defined events without violating the imposed constraints such as the limited battery capacity and taking into account the time window of the events and their priorities.

2.4 Emergency procedure and multi-UAV cooperation

The surrounding environment is dynamic and unexpected events may occur. In this section, an emergency procedure is presented to handle the unexpected events. Afterwards, an iterative approach to manage multi-UAV cooperation is developed to allow UAVs substitute each other smoothly while avoiding redundant use of the resources, e.g. two UAVs covering the same events at the same

time.

2.4.1 Emergency procedure

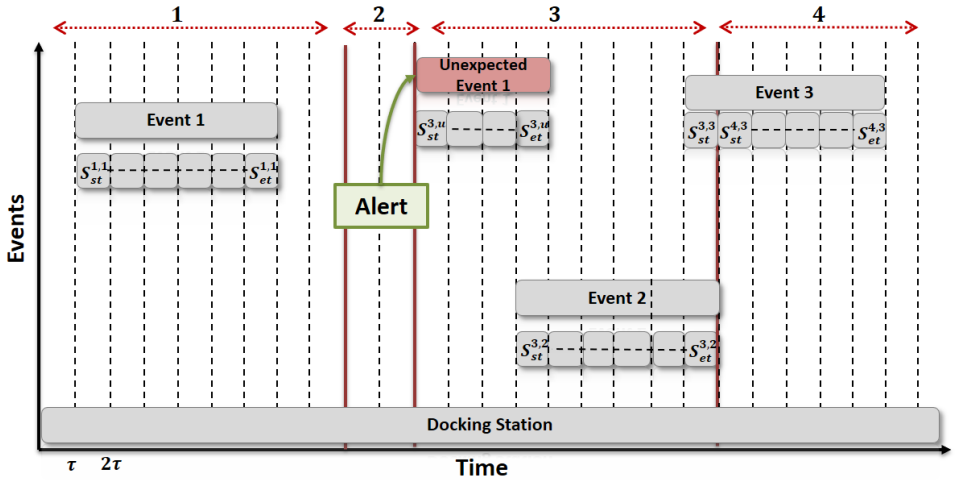


Figure 2.7: Illustration of the new partition in case of the occurrence of an emergency event.

In the previous section, the proposed QL approach is applied when the UAV knows what is happening in the next time period p, which is a proactive autonomous scheduling approach. However, in practice, unexpected events may occur and the UAV is expected to quickly react to handle the emergent environment changes. Hence, in this section, we describe how the QL approach is adapted to the reactive UAV mode. Indeed, the UAV tries to handle the unexpected events once they arise, i.e., once the UAVs are alerted about it. The use of the proposed reward function $\mathcal{R}$ allows the UAV to make the decision to cease covering an event for the benefit of covering a rising emergency thanks to its high priority level. Depending on the current battery energy level, the UAV can either head immediately to attend the emergency in case of enough energy or return to the $\mathcal{DS}$ to sufficiently refill its battery in order to be able to head toward the unexpected event. For this purpose, instead of the fixed time period division referred in Section II as p, we use a dynamic time period division, which is updated whenever an unexpected event is reported. Once happened, even in case

the UAV is floating in the air between two missions, it enters in a standby mode and a new state is created having τ as duration and the current UAV position as location. This new state will serve as a current state for the next period. The set of periods is expressed as: $\mathcal{P}_{dyn} = \{\tilde{p}, \text{where } \tilde{p} = 1, \ldots, P_{dyn}\}$, where P_{dyn} is a dynamic variable. Due to this dynamic division of the time-line, when an unexpected event occurs, the UAV executes the QL described in Section III for the next time division, and updates its schedule accordingly. Hence, the period p in which the UAV was functioning ends at the starting time of the unexpected event and a new division of the rest of the time-line is proposed as depicted in Fig. 2.7.

2.4.2 Multi-UAVs cooperation

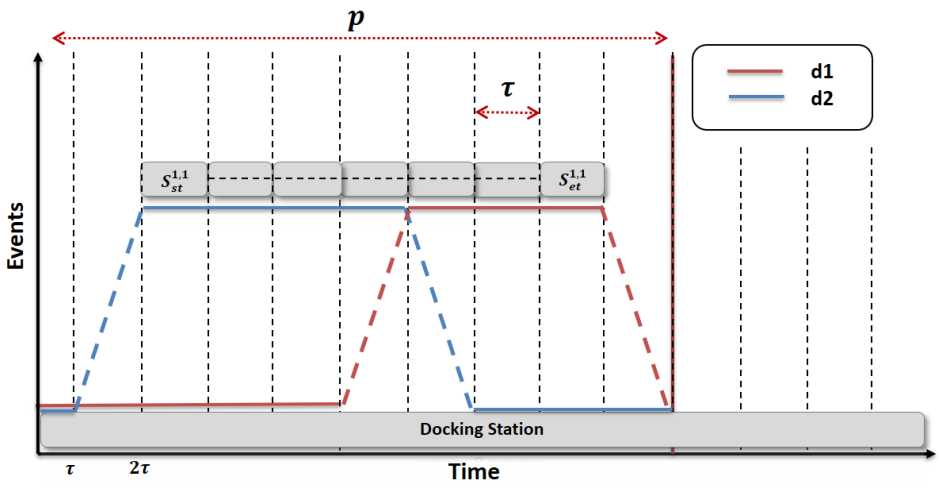


Figure 2.8: Illustration of the iterative approach used by the fleet of UAVs in order to increase the coverage efficiency.

Our objective is to cover the maximum possible of missions, however, the use of a single UAV could not provide the best performance. To this end, we investigate the multi-UAVs system, so that the fleet of UAVs can collaborate and complete missions more efficiently. For instance, one UAV can substitute the other one in case of battery depletion. However, such a process needs to ensure that the substitution is made smoothly without having any time gap in

covering the event. Hence, the substituting UAV must determine when to leave its exact location to reach the target event on time.

In Algorithm 2, we present an iterative approach that provides the framework the capability to handle the unexpected events. In fact, for each period p, one after another, the UAVs attempt to cover the remaining of the missions (line 6-line 9). In the example given in Fig. 2.8, UAV d_1 starts covering the mission, however, its battery capacity is not enough to cover the whole mission. For that reason, UAV d_1 returns to the $\mathcal{DS}$ to refill its battery. Meanwhile, UAV d_2 arrives on time to cover the remaining of the mission. The UAVs keep following the same principle (line 14) if not altered is reported. However, in case of emergency, the UAVs are alerted from the control center of the existence of an unexpected event. Its parameters such as location and priority level are also provided to the fleet (line 12). Hence, the closest UAV with sufficient energy will try to cover the unexpected event. As for the remaining UAVs, their schedules are updated accordingly (as described in Procedure $Update_schedule$). Also, the set of periods $\mathcal{P}_{dyn}$ will be updated accordingly for the remaining time starting from the alert instant. In case of absence of emergency alerts, the UAVs update their schedules normally at the end of each period (Line 21- Line 27). The real-time process is completed when $t = \Gamma$, the length of the makespan (Line 11).

2.5 Data gathering for autonomous UAVs

In this section, we propose the use of the proposed generic approach in the previous sections to create a data gathering mechanism for delay-tolerant wireless sensor networks using autonomous UAVs.

Algorithm 2 Multi-UAVs Cooperation Algorithm and Emergency Procedure

$OL \leftarrow \{d_1, \ldots, d_D\}$ {Set of UAVs}
$\mathcal{P}_{dyn} \leftarrow \{1, \ldots, P\}$ {Set of periods}
$t \leftarrow 0$ {t stands for time}
$p \leftarrow 1$ {p stands for period}
$Al \leftarrow 0$ {Al stands for alert}
for each UAV d in OL **do**
    Update UAV d trajectory using the proposed QL-based approach in Section III for period p
    Update the list of the remaining events to cover in period p
end for
$t \leftarrow t + \tau$
while $t < \Gamma$ **do**
    Check alert and update Al accordingly
    if $Al = 0$ **then**
        UAV d follows the pre-scheduled trajectory
    else
        Get parameters of the unexpected event u reported at time instant t
        $\mathcal{P}_{dyn} \leftarrow Update_schedule(t, u)$
        $Al \leftarrow 0$
    end if
    $t \leftarrow t + \tau$
    if Period p is expired **then**
        $p \leftarrow p + 1$
        for each UAV d in OL **do**
            Update UAV d trajectory using the proposed QL-based approach in Section III for period p
            Update the list of the remaining events to cover in period p
        end for
    end if
end while

Procedure $Update_schedule(Alert_time, u)$
Update $\mathcal{P}_{dyn}$ based on the alert reporting time $Alert_time$
Update OL the order of UAVs based on their distances to the emergency event u (ascending order)
for each d in OL **do**
    Update UAV d trajectory using the proposed QL solution in section III for the new period $\tilde{p}$ in $\mathcal{P}_{dyn}$
    Update the list of the remaining missions to cover in the new period $\tilde{p}$ in $\mathcal{P}_{dyn}$
end for
return P_{dyn}
end Procedure

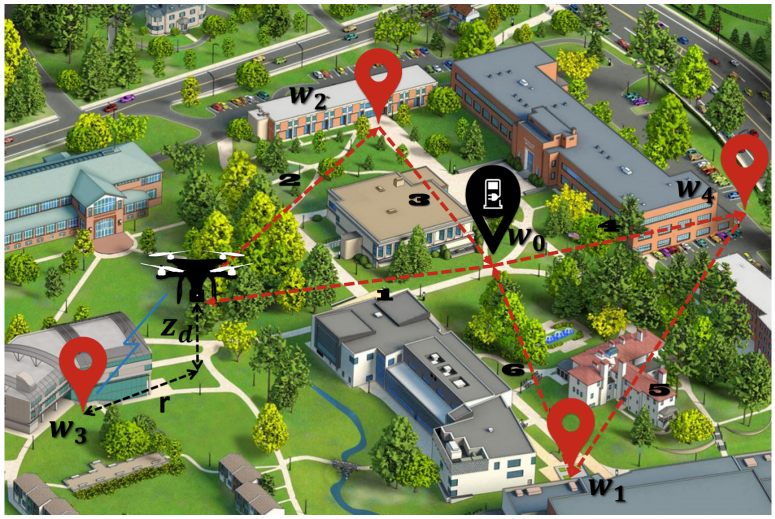


Figure 2.9: Illustration of the autonomous UAV data collection from four sensor nodes $\{w_k, k = 1, \ldots, 4\}$. The UAV starts its journey from the charging station (w_0).

2.5.1 Network model

The set of sensors is defined as $\mathcal{W} = \{w_k, k = 1, \ldots, K\}$, where K is the number of sensors. We assume that the sensing units are equipped with a single unidirectional antenna. We presume that each sensor node w_k has a data reading, denoted by message m_k, intended to be sent to a sink node that could be either a cellular base station or a data collection central unit. The collection of the data from sensors will serve as our missions to cover. Each sensing unit w_k is also defined by the parameters st_k, pt_k, and pr_k which are the acquisition time of data by the sensor (i.e., indicates the moment of availability of the message m_k to be transmitted), the time during which the message m_k should be collected by an UAV, and the message priority, respectively. In practice, different cases of st_k and pt_k values can be encountered depending on the application. For example, the scenario where st_k and pt_k are equal to zero and Γ, respectively, means that the message m_k is ready for collection anytime. Another case could suppose that the message m_k is only available during a certain time window starting at the instant of data acquisition from the sensor and ending after a fixed time period, otherwise the data is lost. In both cases, we are dealing with delay-

tolerant applications. Moreover, in the same way as the previous missions, we assume that each sensing unit is characterized by its 3D geographical location $loc_{w_k} = (x_{w_k}, y_{w_k}, z_{w_k})$. Initially and during periods of inactivity, the UAV is placed at the charging station defined as w_0, located at $loc_{w_0} = (x_{w_0}, y_{w_0}, z_{w_0})$ assimilated as an extra special node, the data gathering central unit, having zero as starting time and Gamma as ending time. Finally, Without loss of generality, we assume that the UAV follows a linear trajectory when moving from a sensing node to another. The autonomous scheduling framework remains applicable for other types of trajectories by measuring the corresponding flying time.

2.5.2 Channel model and achievable rate expression

Due to the nature of the investigated network, we are using an air-to-ground channel type, i.e., we assume $z_{w_k} = 0$, to model the communication link between the UAV and the ground sensors. In this study, we only consider the large-scale path loss effect in the channel gain's expression given as follows :

$$h\left(\Delta(d, w_k)\right) = \frac{1}{\sqrt{PL\left(\Delta(d, w_k)\right)}}, \tag{2.13}$$

where $\Delta(d, w_k)$ is the 3D euclidean distance between the two locations, PL is the path loss in its linear form between UAV d and sensor w_k. The LoS links between the flying unit and the ground sensing nodes are assumed to be available with a certain probability denoted by p^{LoS}. The average air-to-ground free space path loss PL in dB is given as follows [35]:

$$PL_{dB}\left(\Delta(d, w_k)\right) = p_{dB}^{LoS} PL^{LoS} + (1 - p^{LoS}) PL_{dB}^{NLoS}, \tag{2.14}$$

with the non-LoS free path loss PL^{NLoS} and LoS free path loss PL^{LoS} are expressed as follows [35]:

$$PL_{dB}^{X}\left(\Delta(d,w_k)\right) = 10n\log_{10}\left(\frac{4\pi f\Delta(d,w_k)}{C}\right) + L_{dB}^{X}, \tag{2.15}$$

where $X = \{LoS, NLoS\}$, f is the carrier frequency, n is the path loss exponent, C is the speed of light, and L_{dB}^{X} is the average additional loss due to non-LoS/LoS link. The probability of LoS link between the UAV d and the sensor node w_k is given as follows:

$$p^{LoS}\left(\Delta(d,w_k)\right) = \frac{1}{1+\beta_1\exp\left(-\beta_2[\theta\left(\Delta(d,w_k)\right)-\beta_1]\right)}, \tag{2.16}$$

where β_1 and β_2 are constant parameters that depend on the environment and $\theta\left(\Delta(d,w_k)\right)$ is the elevation angle between the UAV d and the sensor node. Evaluating the expression of p^{LoS} shows a trade-off between the path loss and the distance separating the UAV and the sensor node that should be taken into consideration while finding the optimal altitude for the UAV. The higher the UAV's altitude, the lower the path loss. On the other hand, this increases the distance between the flying unit and the ground sensor node which may lead to a lower the transmission rate. In order to compute the required time to transmit a message m_k, denoted by T_k, from a sensor node to the UAV d, we compute the average transmission rate R_{d,w_k}. The transmission time T_k is expressed as $T_k = \frac{M_k}{R_{d,w_k}}$, where M_k is the message size and R_{d,w_k} is computed using the truncated Shannon equation as follows:

$$R_{d,w_k} = \begin{cases} R_{max}, & \text{if } \gamma_{d,w_k} \geq \gamma_{max} \\ B\log_2\left(1+\gamma_{d,w_k}\right), & \text{if } \gamma_{min} < \gamma_{d,w_k} < \gamma_{max} \\ 0, & \text{otherwise,} \end{cases} \tag{2.17}$$

where B is the total bandwidth of the channel, γ_{d,w_k} is the signal-to-interference-plus-noise (SINR) ratio expressed as $\gamma_{d,w_k} = \frac{P|h(\Delta(d,w_k))|^2}{\Omega+N_0B}$, where P is the

ground unit power in Watt with which the signal is transmitted, R_{max} is the maximum possible transmission rate corresponding to γ_{max}. Below γ_{min}, we assume that the signal is not effectively detected and hence, $R_{d,w_k} = 0$. Finally, Ω is the average interference power and N_0 is an additive white Gaussian noise.

In order to determine the UAV stops given the characteristics of the air-to-ground channel and in order to guarantee reliable communication, we solve the following optimization problem to determine the maximum distance separating the stop location of the UAV from the ground sensor:

$$\begin{aligned} &\underset{z_d, r}{\text{maximize}} \quad rd = \sqrt{(x_d - x_{w_k})^2 + (y_d - y_{w_k})^2}, \\ &\text{subject to:} \qquad R_{d,w_k} \geq \eta R_{max}, \end{aligned} \tag{2.18}$$

where rd is the radius between the sensor and the parallel projection of the UAV stop location on the ground 2D plan as shown in Fig. 2.9 and η is a tolerance coefficient regulating the minimum the transmission rate threshold where $0 \leq eta \leq 1$. Hence, rd represents the maximum range of the stop where the UAV needs to hover to collect data from the ground node w_k. In this study, we assume that the exact UAV stop is chosen on the segment connecting the charging station and the node w_k. The optimization problem formulated in (3.18) is only used to determine the UAV stop guaranteeing a communication data rate higher than ηR_{max}. In this sense, the UAV does not need to hover on the top of ground node to collect the data. Reducing η allows the UAV to stop farther but may increase the data transfer time. The optimization problem (3.18) is non-convex due to channel expression but can be sub-optimally solved using numerical or heuristic solutions.

2.6 Navigation for autonomous UAVs

Unlike the previous sections, where we assumed that the UAV follows a linear trajectory when moving from a mission to another, in this section, we propose an autonomous UAV path planning framework using deep RL approach. The objective of this section is to create a framework that guides the UAV to reach autonomously spatially distributed moving or static targets in its given three dimensional urban area. In this approach, the DDPG algorithm, described in Chapter 2, is employed to train the UAV to navigate through or over the obstacles to reach its assigned target with continuous action space. A customized reward function is developed to minimize the distance separating the UAV and its destination while penalizing collisions.

2.6.1 Urban area model

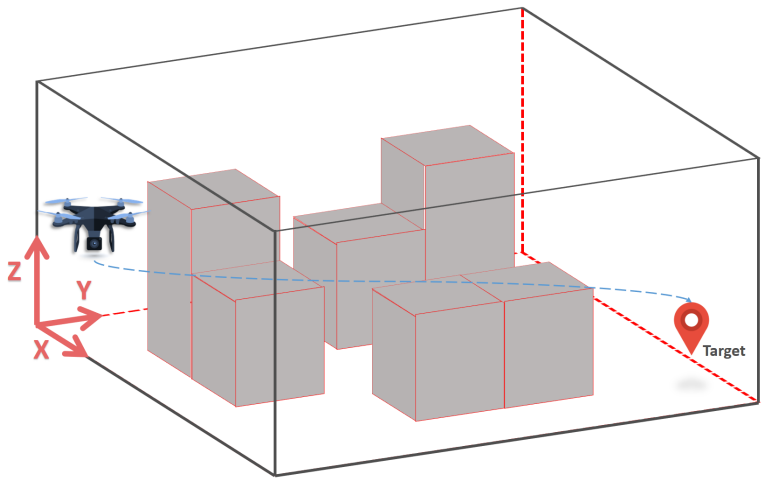


Figure 2.10: Illustration of the autonomous obstacle-aware UAV navigation in an urban environment.

Autonomous navigation for UAVs in real environment is complex. Hence, Without loss of generality, we create a virtual 3D environment with high matching degree to the real-world urban areas. Unlike most of the existing virtual environments, which are studied in literature and usually modeled as a grid world, in this paper, we focus on a free space environment containing 3D obstacles that

may have diverse shapes as illustrated in Fig. 2.10. Consequently, the UAV has the freedom to take any direction and speed to reach its target unlike grid world, which restricts the freedom of UAV into a finite set of actions. The goal is to train the UAV to fly safely from any arbitrary starting position to reach any destination in the considered area with continuous action space. The investigated system assumes the following assumptions:

- The environment obstacles have different heights. Each one of them is represented by a 3D polygon characterized by its the starting point [x_{obs}, y_{obs}], the set containing the edges of the base edg_{obs}, and its height h_{obs}. Hence, if having an altitude higher than the obstacle's height, the UAV can cross over the obstacles. Otherwise, the UAV can avoid it by flying around.

- The destination location is known to the UAV and it can be either static or dynamic (i.e., the target location can evolve over time). If the destination location is dynamic then it follows a random pre-defined trajectory, that is unknown by the UAV.

2.6.2 UAV action space

For each taken action, we assume that the UAV chooses a distance to cross according to a certain direction in the 3D space during Δt units of time. The action is modeled using the spherical coordinates (ρ, ϕ, ψ) as follows:

$$x_d = x_d + \rho \sin\phi \cos\psi, \tag{2.19a}$$

$$y_d = y_d + \rho \sin\phi \sin\psi, \tag{2.19b}$$

$$z_d = z_d + \rho \cos\phi, \tag{2.19c}$$

where ρ is the traveled radial distance by the UAV in each step ($\rho \in [\rho_{min}, \rho_{max}]$), where ρ_{max} is the maximum distance that the UAV can cross during the step

length Δt. Its value depends on the maximum speed of the UAV denoted by v_{max}. The parameter ψ denotes the inclination angle ($\psi \in [0, 2\pi]$), and ϕ represents the elevation angle ($\phi \in [0, \pi]$). For instance:

- if $\rho = \rho_{max}$, $\phi = \pi$, and any value of ψ, the UAV moves by ρ_{max} along the Z axis.
- if $\rho = \rho_{max}$, $\phi = \pi/2$, and $\psi = 0$, the UAV moves along the x axis.

The UAV coordination and the target coordination are used as the input for our DDPG model. While, the output of the framework is the tulip (ρ, ϕ, ψ), with the objective to guide the UAV to reach its destination safely.

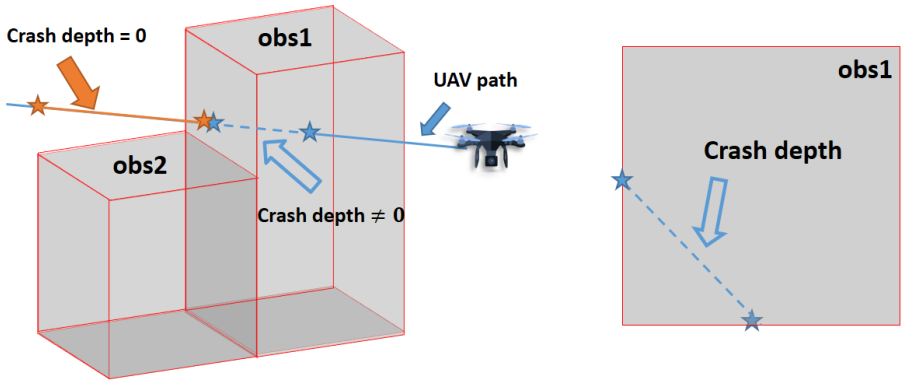


Figure 2.11: Example representing a collision scenario. The UAV's altitude is less than the obstacle's height (obs1). The action is chosen such that the UAV crosses the obstacle. This results in an obstacle penalty reflecting the underwent depth.

2.6.3 Reward function

In an obstacle-constrained environment, the UAV must avoid obstacles and autonomously navigate to reach its destination in real-time. Therefore, the reward function, denoted by f_r, is modeled such that it encourages the UAV to reach its destination and, at the same time, penalizes it when crashing. Thus, the reward function is composed of two terms: target guidance reward and obstacle penalty. The target guidance reward, denoted by f_{gui}, is used to motivate

the flying unit to reach its target as fast as possible, while the obstacle penalty, denoted by f_{obp} is responsible for alerting the UAV to keep a certain safety distance off the obstacles. The reward function is formulated as follows:

$$f_r(\Delta(d, w_k), \sigma) = (1 - \beta) f_{gui}(\Delta(d, w_k)) + \beta f_{obp}(\sigma), \tag{2.20a}$$

$$f_{gui}(\Delta(d, w_k)) = \exp(-5\Delta(d, w_k)^2), \tag{2.20b}$$

$$f_{obp}(\sigma) = \exp(-100\sigma) - 1, \tag{2.20c}$$

where σ is the crash depth explained in Fig. 2.11 and β is a variable that regulates the balance between f_{obp} and f_{gui}. The obstacle penalty is modeled as a function of the crash depth σ to conserve the continuous nature of the reward function instead of using discrete penalty, which proved to be more efficient to help the model to converge. In fact, when the crash depth is high, the UAV receives a higher penalty, whereas a small crash depth results in a lower penalty. The use of this approach helps the UAV learn efficiently over the training episodes how to adjust its trajectory to avoid obstacles.

2.6.4 DDPG and transfer learning

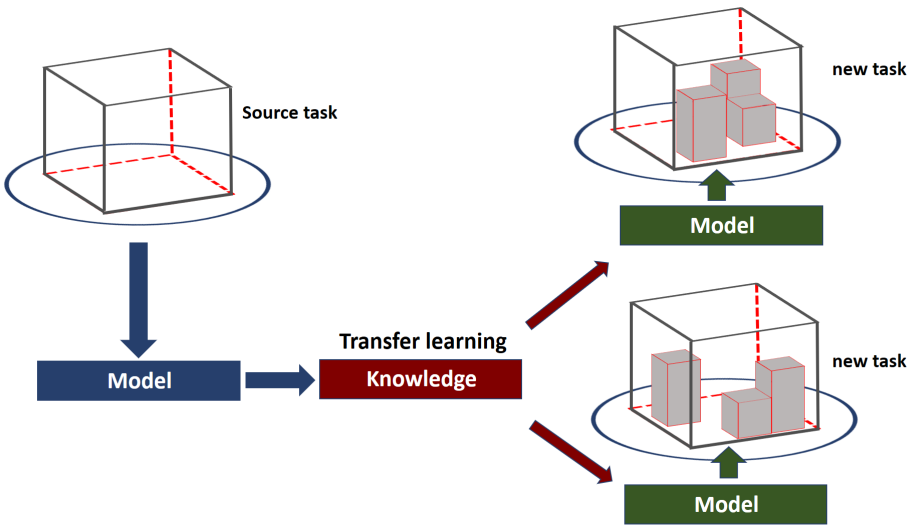


Figure 2.12: Illustration of the transfer-learning technique.

Transfer learning is a machine learning technique used to transfer the knowledge to speed up training and improve the performance of deep learning models. The proposed approach to train the UAV consists in two steps. Initially, we train the model in an obstacle-free environment. Training in such environment, grants the UAV the capability to reach any target in the covered 3D area with continuous space action. Then, the trained model on the obstacle-free environment will serve as a base for future models trained on other environments with obstacles. Afterwards, we transfer the acquired knowledge (i.e. source task) and use it to improve the UAV learning of new tasks where it updates its path based on the obstacle locations while flying toward its target. The adopted transfer learning technique applied to DDPG for autonomous UAV navigation is illustrated in Fig. 2.12. Once the training phase is completed offline, the UAV is capable to make instant decisions, while interacting with the environment, to manage real-time missions.

2.7 Summary

In this chapter, we presented the system models for our autonomous UAVs and the adaptation of RL algorithms to our models. Also, the states and the actions are thoroughly discussed for our different scenarios while including several constraints.

Chapter 3

Simulation and results

3.1 Introduction

In this chapter, we study the behavior of the autonomous UAVs for various selected scenarios and environments.

3.2 Evaluation for UAV scheduling

In this section, we study the behavior of the system for various selected scenarios. We also visualize the impact of dividing the time span into P sub-period in executing the assigned missions and evaluate the convergence of the RL algorithm. To do so, we assume that M tasks having randomly generated locations within 5×5 km^2 area. An example is provided in Fig.3.1. The origin point is located at $loc_0 = (2.5, 2.5, 0)$. The tasks take place all along a makespan $\Gamma = 15000$ seconds. Their duration are uniformly generated between 10 and 25 minutes. a Table 3.3 presents the simulation parameters used such as $\bar{b}$ [36], the battery maximum capacity and V_d the UAV average speed. The simulations are executed using Matlab. First, we investigate two scenarios where a single UAV, with unlimited battery capacity, has to cover the maximum possible of missions as illustrated in Fig. 3.2. In other words, in those scenarios, we will not consider the limited battery constraint to visualize the behavior of the UAV to cover the

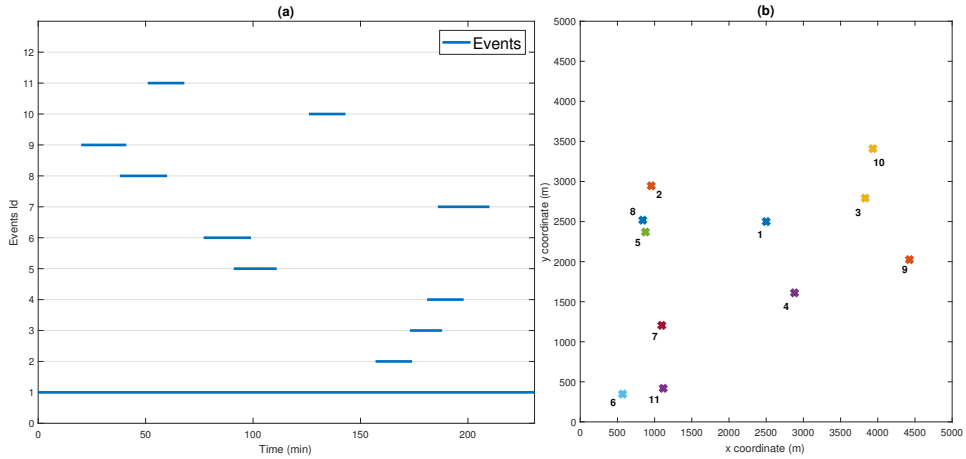


Figure 3.1: Studied case with $E = 10$ pre-planned events (a) temporal distribution in 230 min and (b) event locations in 5×5 km^2.

spatially and temporally distributed events. After that, we will show the impact of use of the time line division on the framework results.

In the first scenario, denoted by Scenario A, we consider non-overlapping tasks so we took off missions 8, 6, 3 and 4 from the original scenario given in Fig.3.1. Hence, the UAV needs to cover the remaining $M = 6$ missions. Fig. 3.2 shows the behavior of the UAV for Scenario A where it successfully covers 100% of the missions. Initially, it stayed at the $\mathcal{DS}$ until before 4 minutes to move to task 4. Then, it returned to the $\mathcal{DS}$ as there no active mission. Later, the UAV keep covering the rest of the events in a similar way. The UAV successfully learn to respect the time distribution of the events and preserve sufficient time to reach the events on time, i.e, take off sufficiently early to reach the event on time.

Table 3.1: Simulation parameters for the scheduling framework

Parameter	Value	Parameter	Value
α	0.5	γ	0.4
ϵ_{min}	0.01	ϵ_{max}	1
d	0.002	$\Gamma(sc)$	15000
$V_d(m/s)$	10	$\tau(sc)$	60
b_{init}	0	$\bar{b}(mAh)$	160000
P	3	n	3

In Scenario B presented in Fig. 3.3, we determine the activity plan of the

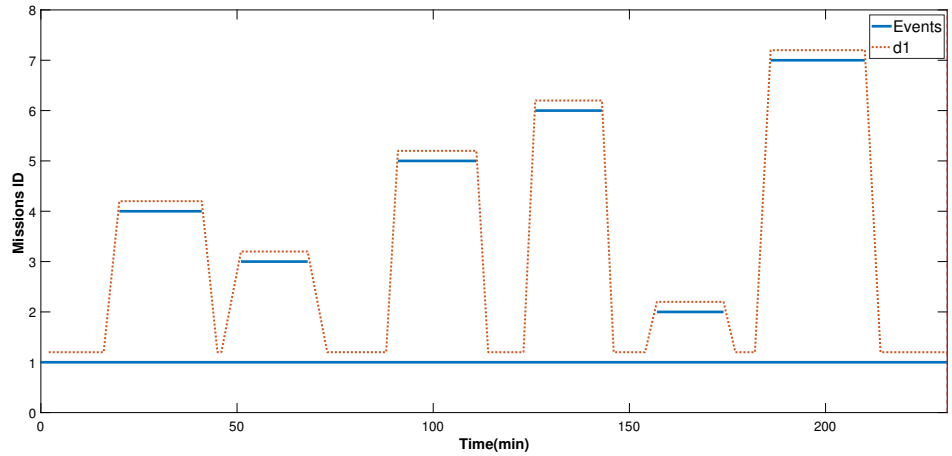


Figure 3.2: Scenario A: Autonomous scheduling for $E = 6$ non-overlapping tasks.

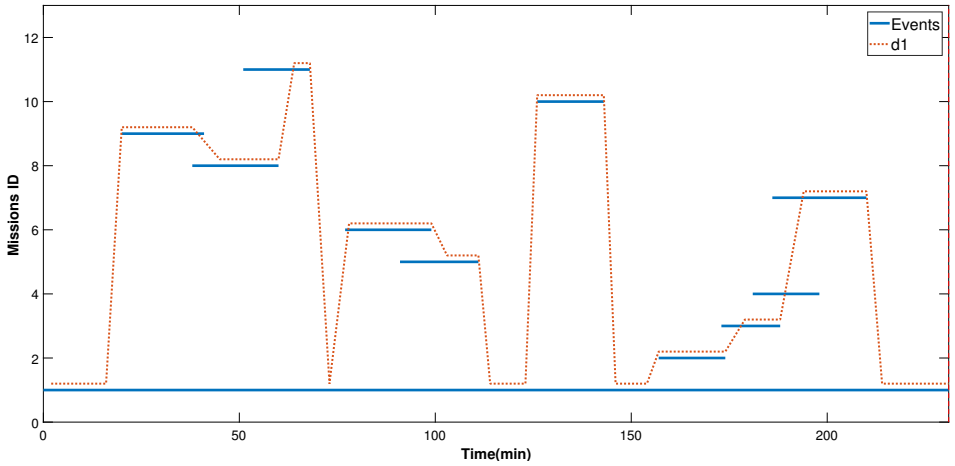


Figure 3.3: Scenario B: Autonomous scheduling for $E = 10$ overlapping tasks.

UAV considering all the overlapping missions. The UAV tries to cover the maximum of events. In this case, the UAV may decide to ignore a mission and go to the other, to cover the longest event, i.e., higher reward. For instance, the UAV leave event 3 and move to event 7 instead of event 4 despite the fact that event 4 starts before task 7 and it is closer to event 3 than event 7. This is explained by the fact that task 7 remaining time is longer than the event 4. Indeed, it is more advantageous to cover task 7 instead of spending extra time in flying between the tasks. Minute 75, shows the fact that the UAV should return to $\mathcal{DS}$ in case of inactive missions. This scenario shows that the UAV successfully learned how to cover the missions having highest scores. In this scenario, the total coverage

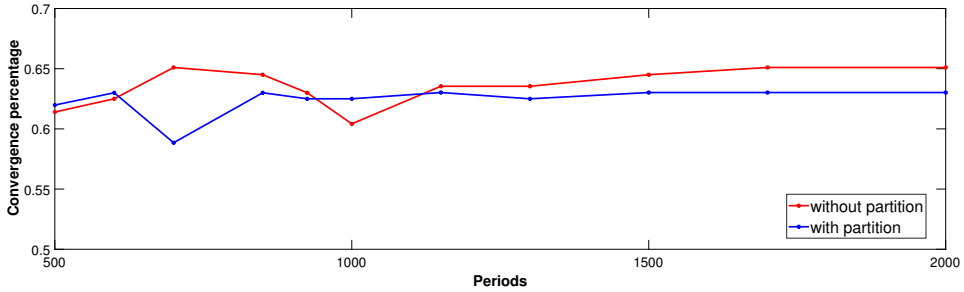


Figure 3.4: Convergence speed of the proposed QL-based approach versus the number of training episodes with $P = 3$ periods.

percentage is about 65%.

As for Fig. 3.4, we present the impact of the period division on the convergence speed of our proposed solution. The blue line represents the coverage percentage with the use of the time line division where $P = 3$ periods, i.e. limited UAV knowledge about the future. The blue line indicates that the algorithm converges approximately at 750 episodes with a maximum coverage percentage about 63%. The red line represents the coverage percentage without the use of the time line division, i.e. full UAV knowledge about the future. It indicates that the algorithm converges approximately at 1750 episodes with a maximum coverage percentage about 65%. Therefore, with the division of the time line into periods, we compromise the efficiency in order to gain speed, which is a crucial factor that we will need to deal in "real-time" with the unexpected events later.

Next, we study two scenarios where we take into consideration the limited battery capacity of the UAV. In the first scenario, depicted in Fig. 3.5, without the use of the P periods division, the UAV managed the cover about 29% of the total events. As shown in Fig. 3.5 (b), where the evolution of the UAV battery during the execution of the missions is presented, the UAV always returns to the $\mathcal{DS}$ in case of low level of power to avoid crashes and, at the same time, ensure better coverage for better coverage of events. In addition, the UAV tries to return to $\mathcal{DS}$ as its battery level reaches the blue dots, which indicates the minimum energy required in each period in order to cover the farthest event and that will

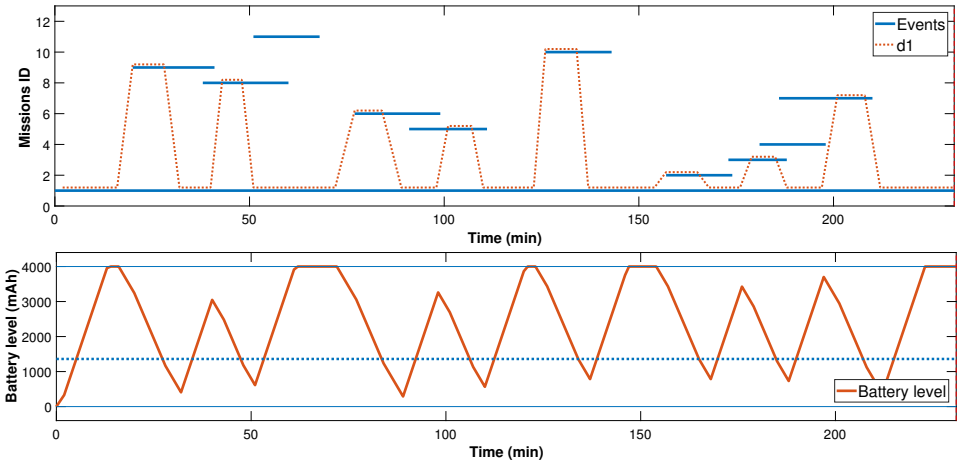


Figure 3.5: Scenario C: (a) autonomous scheduling with limited battery capacity $b = 4000$ mAh, with $P = 1$ period and $K_{ep} = 1500000$, and (b) the evolution of the UAV battery level during the execution of the missions. The continuous blue lines indicate the boundaries of the UAV battery capacity, as for the blue dots, they indicate the maximum energy required in each period in order to cover the farthest event.

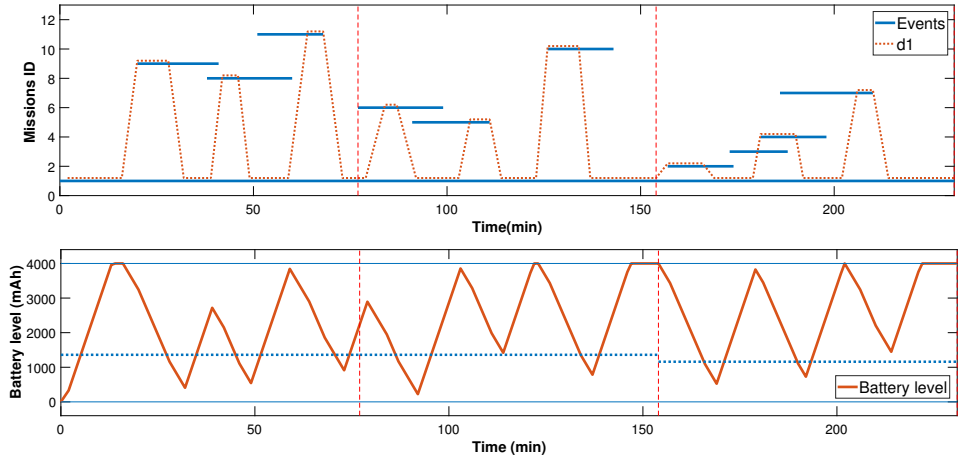


Figure 3.6: Scenario D: (a) autonomous scheduling with limited battery capacity $b = 4000$ mAh, with $P = 3$ periods and $K_{ep} = 200000$ per period, and (b) the evolution of the UAV battery during the execution of the missions. The continuous blue lines indicates the boundaries of the UAV battery capacity, as for the blue dots, it indicates the maximum energy required in each period in order to cover the farthest event.

serve as a safety margin for our UAV. In scenario D, depicted in Fig. 3.6, with the use of the $P = 3$ periods, the UAV succeed to cover about 28% of the total events. As mentioned earlier, the framework loses some coverage efficiency in favor of obtaining faster results. This scenario shows that the UAV successfully learned how to cover the missions while respecting the battery boundaries, no

crashes. As for the choice of the P periods, the rapidity of the framework, may differ depending on the application. In our case, we aim to achieve a suitable framework for real time applications using UAVs, thus, we choose the partition model. Furthermore, there is another advantage of the use of the time line division. This will be shown in the next scenario, in case of occurring of an unexpected event. As shown in Fig.3.7 and tion IV-A, with the occurrence of an expected event, which is set as a higher priority event, the UAV leaves event 10 midway in order to cover event 12, colored in green. Also, the framework updates the dynamic set of periods, that we denoted by $\mathcal{P}_{dyn}$, by ending the current period at time of occurrence of the emergency and starting a new re-partition over the time left. The framework was able to reschedule the UAV's path in a faster manner by considering only the events during the time period instead of the whole remaining events. With limited knowledge, the UAV can make a faster decision and adapt its trajectory quicker in case of emergency. In Fig. 3.8, we present the obtained coverage percentage versus the UAV battery capacity $\bar{b}_d$ for Scenario D. We notice that while increasing $\bar{b}_d$, the coverage increases too. This is explained by the fact that increasing $\bar{b}_d$, helps the UAV spend more time in covering the events and reduce the required number of travels to $\mathcal{DS}$ to refill

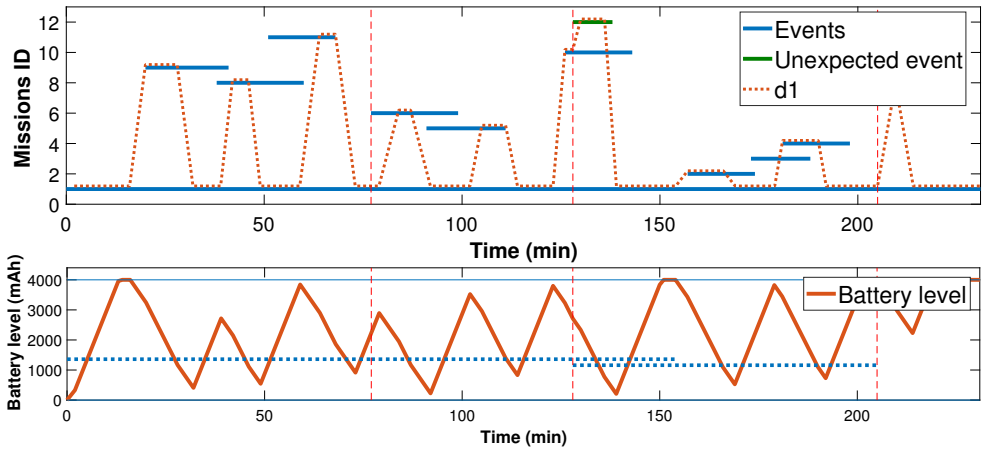


Figure 3.7: Scenario E: (a) autonomous scheduling with limited battery capacity $b = 4000$ mAh, with the occurrence of an unexpected event, and (b) the evolution of the UAV battery during the execution of the missions. The green colored event denotes the unexpected event.

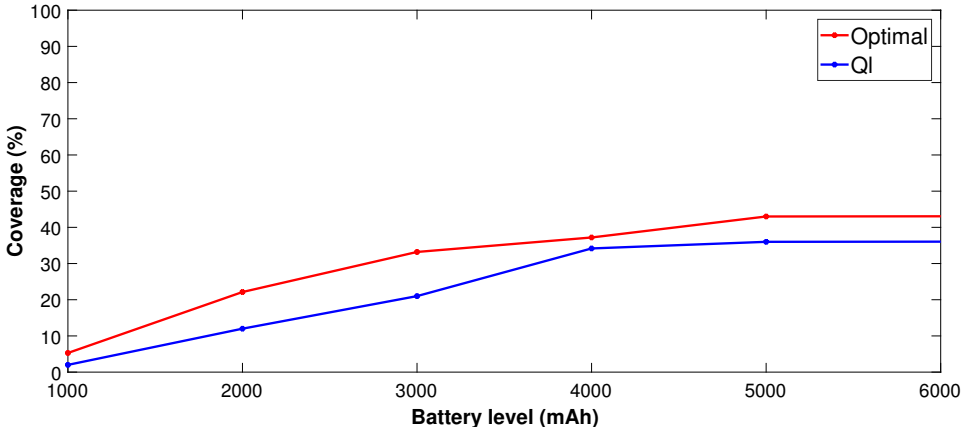


Figure 3.8: Coverage percentage of a sole UAV in function of battery level, red line refers to optimal solution, blue line refers to the proposed QL approach.

the battery. The figure also compares the performance of the QL approach with an optimal MILP based solution proposed in [13]. The QL solution achieves satisfactory results known that the proposed solution is compromising the efficiency in order to gain speed, which is a crucial factor that optimal solution can not provide. Though, it is essential be able to deal with emergencies in real time.

3.3 Evaluation of the multi-UAVs cooperation

In the next scenario, we will use a fleet of UAVs where $D = 3$. As shown in Fig. 3.9 and explained in Section 3.4, one after another, the UAVs attempt to cover the remaining missions. In case of insufficient energy to cover the event, other UAVs will play the role of relief UAV trying to cover the remaining part of the event. For example, the UAVs successfully cover the whole of event 9 without any delay. The three units complement each other, without overlapping, to efficiently cover the whole event 9 using the proposed Multi-UAVs cooperation algorithm. Each UAV, respecting a certain order, tries to autonomously determine the best schedule to cover the maximum possible of the remaining events. Then, it forwards its plan to the next UAVs in order to do the same process,

while synchronizing its plan with the forwarded schedule.

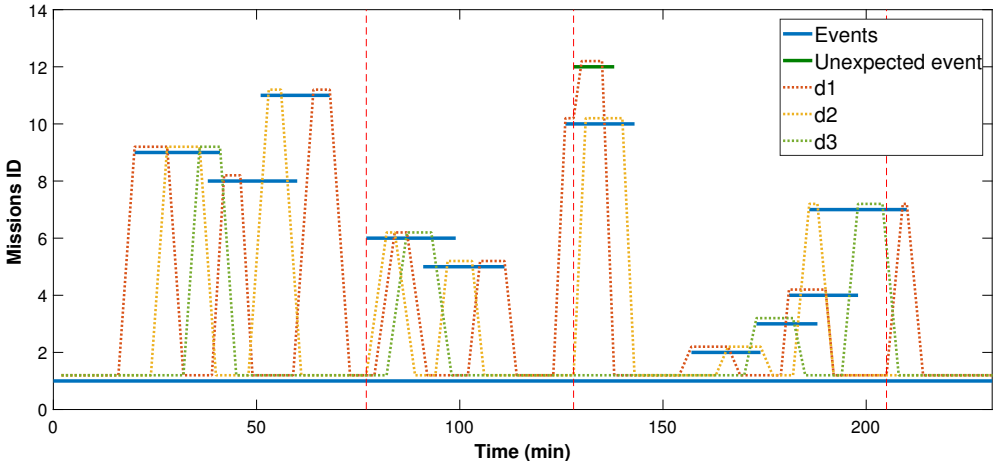


Figure 3.9: Scenario F: autonomous scheduling using a fleet of UAVs $(D = 3)$ with limited battery capacity $b = 4000$ mAh and the occurrence of an unexpected event.

Table 3.2: UAV activity plan for Scenario F

UAV	Trajectory	coverage percentage
d1	$\mathcal{DS} \rightarrow 9 \rightarrow \mathcal{DS} \rightarrow 8 \rightarrow \mathcal{DS} \rightarrow 11 \rightarrow \mathcal{DS} \rightarrow 6 \rightarrow \mathcal{DS} \rightarrow 5 \rightarrow \mathcal{DS} \rightarrow 10 \rightarrow 12 \rightarrow \mathcal{DS} \rightarrow 2 \rightarrow \mathcal{DS} \rightarrow 4 \rightarrow \mathcal{DS} \rightarrow 7 \rightarrow \mathcal{DS}$	0.27
d2	$\mathcal{DS} \rightarrow 9 \rightarrow \mathcal{DS} \rightarrow 11 \rightarrow \mathcal{DS} \rightarrow 6 \rightarrow \mathcal{DS} \rightarrow 5 \rightarrow \mathcal{DS} \rightarrow 10 \rightarrow \mathcal{DS} \rightarrow 2 \rightarrow \mathcal{DS} \rightarrow 7 \rightarrow \mathcal{DS}$	0.20
d3	$\mathcal{DS} \rightarrow 9 \rightarrow \mathcal{DS} \rightarrow 6 \rightarrow \mathcal{DS} \rightarrow 3 \rightarrow \mathcal{DS} \rightarrow 7 \rightarrow \mathcal{DS}$	0.14

However, it might be possible that the UAVs are not able to cover the whole events. Indeed, the duration of the predefined events might be too long compared to their battery life time i.e. the used UAVs are capable to only fly for 25 minutes without executing any tasks. In addition, several events are overlapping which make it impossible for only three UAVs to simultaneously cover them. This issue can be resolved by employing more UAVs, increasing their battery capacity, or dealing with a reduced tasks' time windows. The trajectory of each UAV for this scenario is presented in Table II. As described in Algorithm 2, the UAVs follows a certain order. This justify the fact that the UAV d1 coverage percentage is higher than the one of UAVs d2 and d3: they are are operating as a relief for the UAV d1.

In case of an unexpected event, the closest UAV will try to cover the whole unexpected event. In minute 130, the UAVs are alerted about the unexpected event, colored in green. At that time, UAV d1 was covering event 10 and both

UAVs 2 & 3 were at the docking station. As shown in Fig. 3.1(b), event 10 is closer to the emergency, denoted as event 12, than $\mathcal{DS}$. Hence, UAV d1 ceased covering event 10 and headed directly toward the event 12. Alternatively, UAV d2 replaced UAV d1 in covering the remaining of event 10.

3.4 Evaluation of UAV data gathering

In this section, we study the behavior of the system for data gathering scenarios. Also, a comparison between the performance of the QL-based approach and an optimal MILP-based solution is provided [37]. To do so, we assume that $K = 5$ sensing nodes are randomly generated locations within 5×5 km^2 area. The charging station w_0 is located at $loc_{w_0} = (2.5, 2.5, 0)$. The data collection takes place all along a makespan $= 60$ minutes. The power model values are obtained from Table I of [34]. We investigate the scenario where the UAV has to collect the maximum possible of messages from the scattered sensor nodes as illustrated in Fig. 3.10. In Scenario G, we consider $K = 5$ sensor nodes, having message size $M_k = 10$ Mo to cover. The nodes are assumed to send a high amount of data, e.g., recorded videos or all data collected from neighborhood sensors. Fig. 3.10 depicts the behavior of the UAV for Scenario G where it successfully collects the data from all sensors in 38 minutes. At the beginning, the UAV stays at the charging station w_0 until minute 7.3 to charge its battery till 2310 mAh, initially empty, in order to have enough energy to start roaming. Therefore, the flying unit heads toward sensor nodes w_5, w_1, and w_2, respectively. Then, it returns to the charging station, situated at loc_{w_0}, to charge its battery till it has enough energy to collect messages from the remaining sensor nodes w_3 and w_4 before returning to the charging station. The UAV successfully learned how to collect all the messages from the sensing nodes while respecting the battery limit. In Scenario H (Fig. 3.11), we associate to the sensor w_3

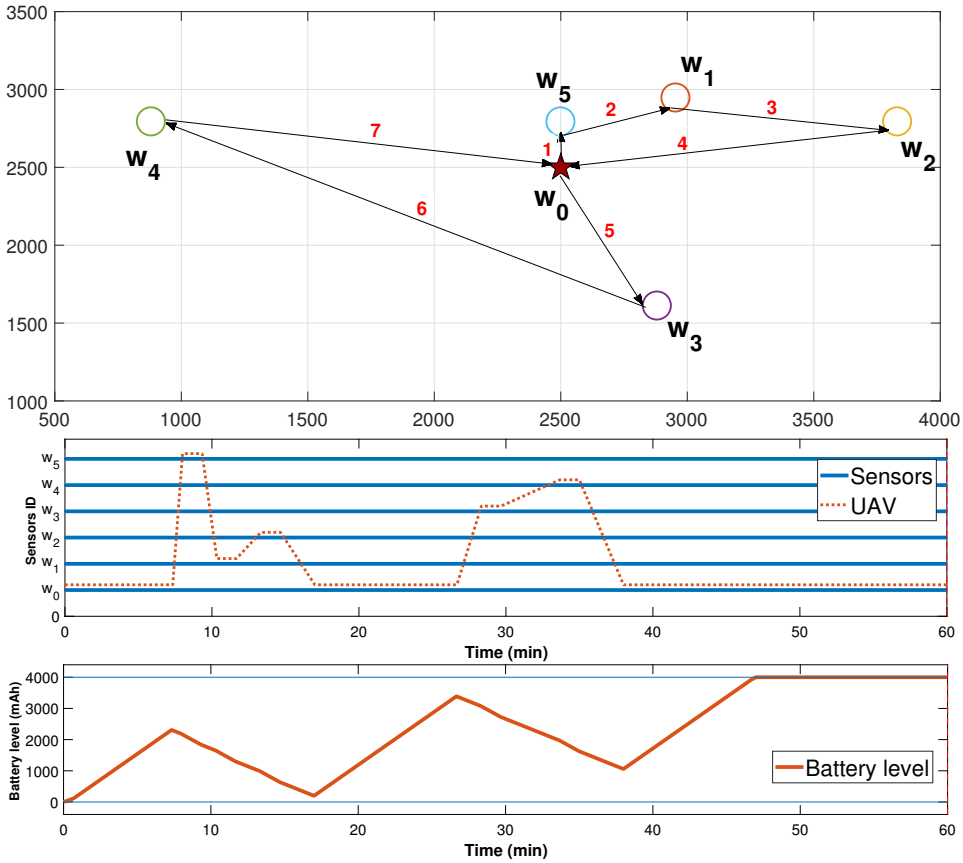


Figure 3.10: Scenario G: (a) UAV data collection path plan for five spatially distributed sensing nodes located in 5×5 km^2, (b) autonomous scheduling with limited battery capacity $\bar{b} = 4000$ mAh, and (c) the evolution of the UAV battery during the UAV trip.

a higher priority level than its peers. Hence, the UAV adjusts its trip plan compared to Scenario G. The UAV stays for 5.33 minutes at the charging station till it has enough energy 1650 mAh to head toward sensor w_3 to cover it first, as it has highest priority, then covers sensor w_5. Afterward returns to w_0 to charge its depleted battery. Next, the flying unit covers sensors w_1, w_2, and w_3, respectively, with a return to the charging station in-between to avoid crashes. The UAV successfully learned how to collect all messages from the sensors while respecting the priority of the sensors and the battery limit.

In Fig. 3.12, we present the required time to cover all the sensor nodes versus the ground unit transmission power P for Scenario G and another Scenario I, where the messages of sensors w_3 and w_4 are reduced to $M_k = 4$ Mo. We notice that for both scenarios, decreasing P leads to higher data collection time. This is

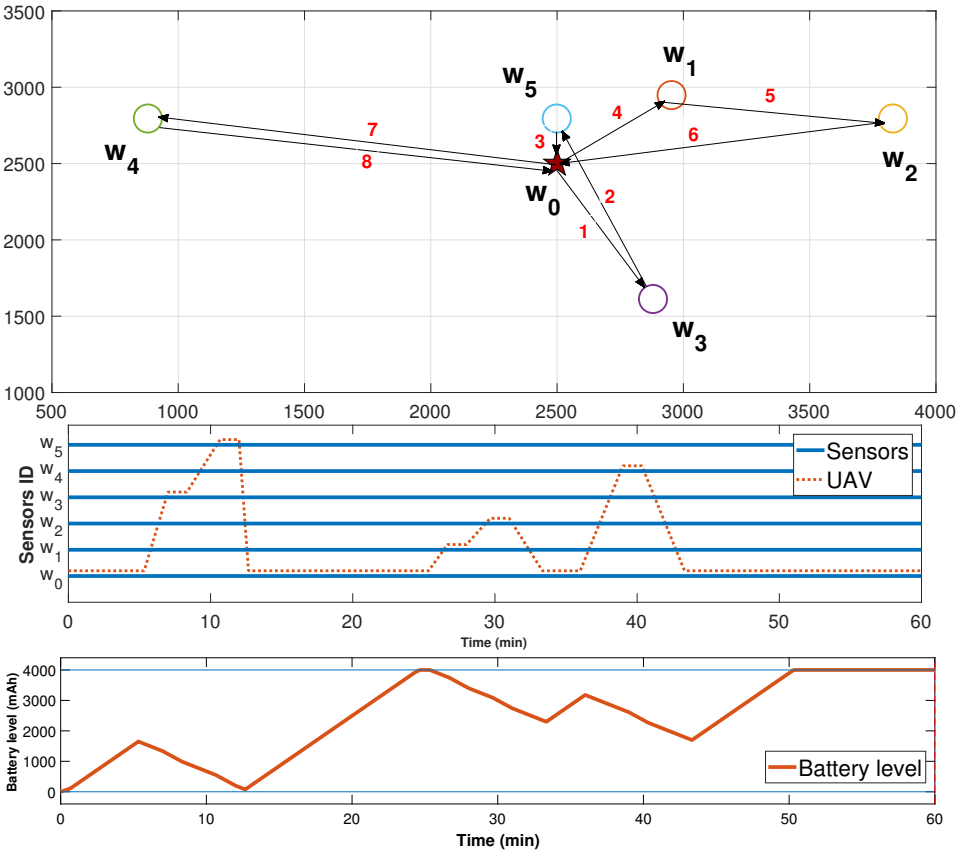


Figure 3.11: Scenario H: UAV data collection path plan, (b) autonomous scheduling with limited battery capacity $\bar{b} = 4000$ mAh, and (c) the evolution of the UAV battery during the UAV trip. In this Scenario, sensor node w_3 has a priority level higher than its peers.

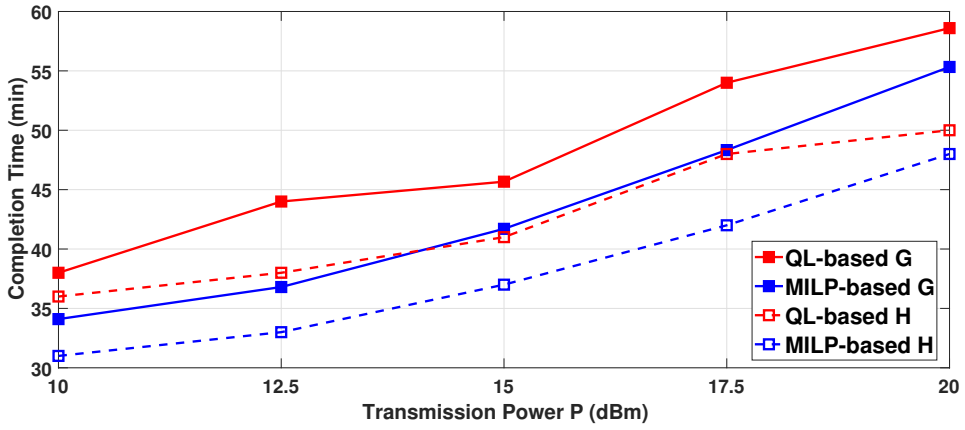


Figure 3.12: Tour completion time of the UAV as a function of the ground unit transmission power P for two scenarios G and I. Blue line: MILP-based optimal solution and red line: proposed QL approach.

due to not only the decrease of the transmission rate but also to the more frequent battery reload since more hovering energy is needed. It is also shown that, in Scenario I, decreasing the message size makes the data collection procedure

faster and that is due to lower transmission time. The figure also compares the performance of the QL approach with an optimal MILP based solution, i.e., a learning technique versus a deterministic one. The QL solution achieves close result to the optimal with about 10% difference in the tour completion time on average. The behavior of the autonomous UAV for the investigated scenario corroborates the ability of QL to handle the data gathering problem efficiently.

3.5 Evaluation of UAVs Navigation

In this section, we study the behavior of the autonomous navigation of UAVs in urban areas. We also visualize the efficiency of the framework in terms of crash rate and tasks accomplishment. To do so, we assume that the UAV starting location loc_d, its target location loc_e, and the obstacles' parameters are randomly generated within a cube-shaped area with 100 m edge length. We make sure that the locations of both the targets and the UAV are outside the obstacles. Table 3.3 presents the simulation parameters of the investigated environment and used for the DDPG training. The simulations are executed using Python.

Table 3.3: Simulation parameters for the navigation framework

Parameter	Value	Parameter	Value	Parameter	Value
β	4	ν	0.99	ρ_{max} (m)	0.2
T	100	M	40000	ϵ_{end}	0.1
B	10000	N	256	ϵ_{start}	0.9

For the sake of clarity, the figures concerning the UAV path planning are presented in only 2D dimension area (i.e., plan(x,y)) and we provide beside each dot, the altitude of either the target or the UAV. In the first scenario, we consider an obstacle-free environment. The destination location is assumed to be dynamic, that it keeps moving in a randomly generated way. As shown in Fig. 3.13, the UAV is successfully adapting its trajectory based on the location

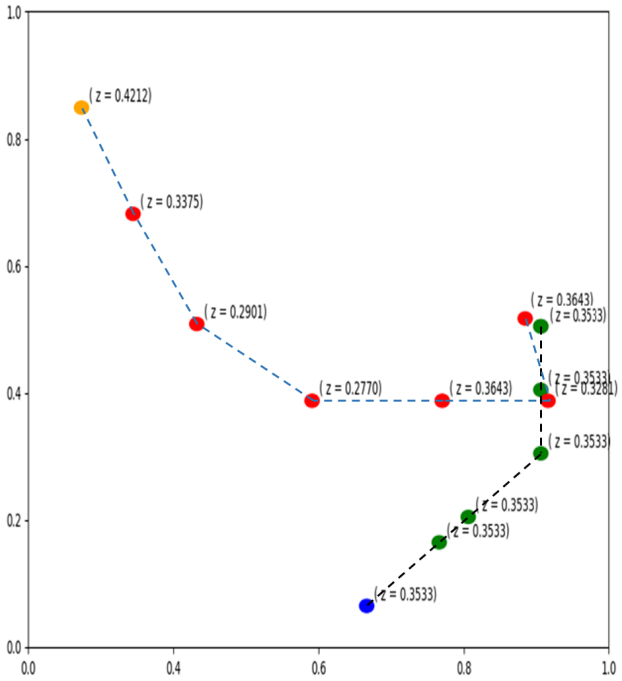


Figure 3.13: Path followed by UAV based on the dynamic target location. Yellow dot: initial UAV position, blue dashed line: trajectory of the UAV, blue dot: target initial location, and dashed black line: the target trajectory.

of its target until it reaches it. This shows that the UAV succeeded in learning how to update each direction in order to "catch" its assigned destination.

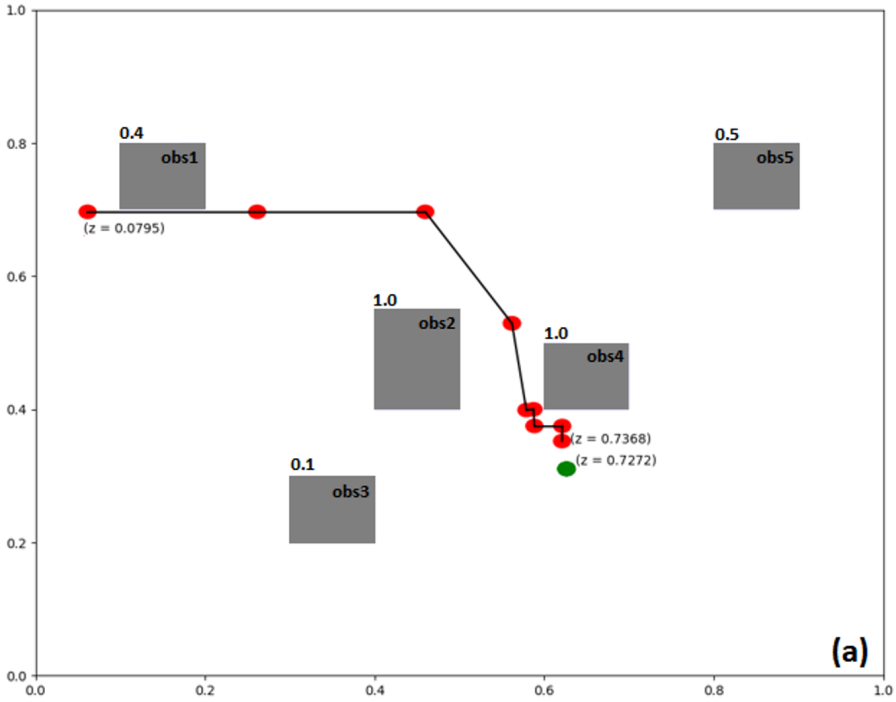


Figure 3.14: Path followed by UAV based on the static target location for environment 1 (env1). Red dots refers to the UAV, green dot refers to the target, and gray boxes to the obstacles.

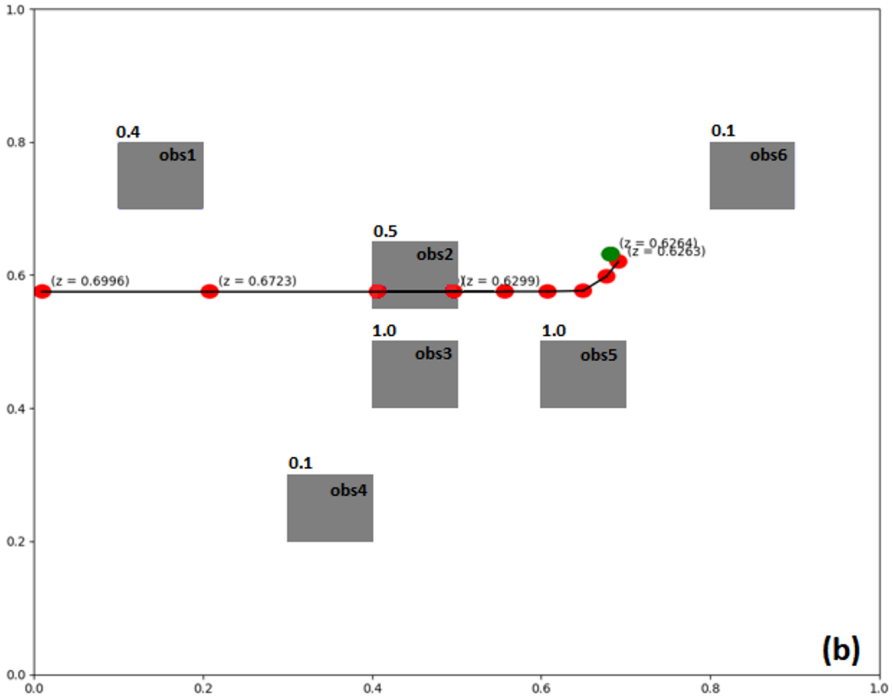


Figure 3.15: Path followed by UAV based on the static target location for environment 2 (env2). Red dots refers to the UAV, green dot refers to the target, and gray boxes to the obstacles.

In the next scenarios, the obstacles are added in a random disposition with different heights as shown in Fig.[3.14, 3.15, 3.16]. In these cases, we assume that the target destinations are static. In Fig. 3.14, the UAV successfully reached its destination location while avoiding the obstacles. In Fig. 3.15, on its way to the destination, the UAV crossed over $obs2$ ($z_u = 0.63 > h_{obs2} = 0.5$) in order to reach faster its target location unlike the case in Fig. 3.14, where the UAV could not cross over $obs2$ to reach its destination as soon as possible because of the obstacle height (maximum height). In Fig. 3.16, having a higher altitude than $obs6$, the UAV crossed over $obs6$ to reach its target. In all cases, scenarios show some lacking in precision to reach the target location due to the fact of using infinite action space which makes it hard to get pinpoint accuracy. These scenarios showed that the UAV successfully learned how to avoid obstacles to reach its destination. In Fig. 3.17, we present the reward received by the UAV during its training phase, in Fig. 3.14 shows that the UAV learns to obtain the maximum reward value in an obstacle-free environment. We successfully

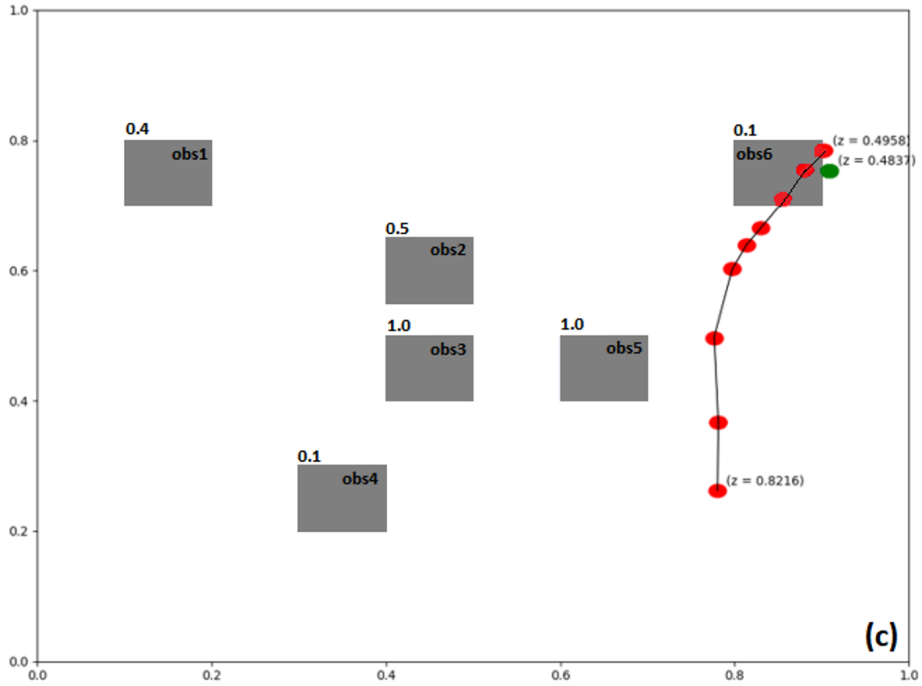


Figure 3.16: Path followed by UAV based on the static target location for environment 2 (env2). Red dots refers to the UAV, green dot refers to the target, and gray boxes to the obstacles.

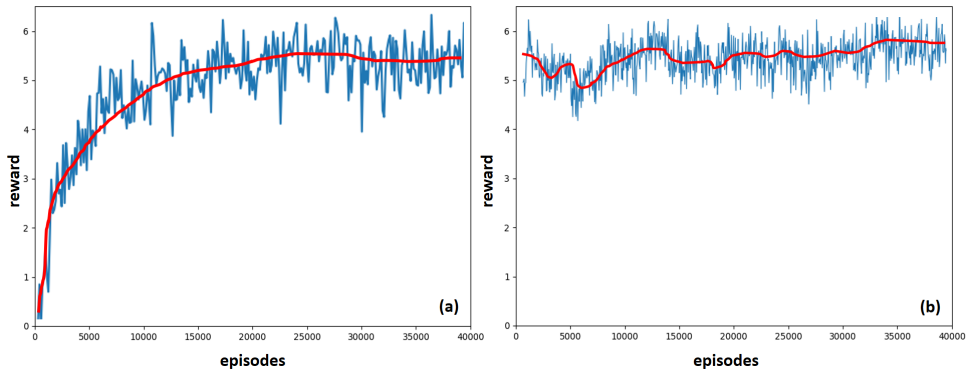


Figure 3.17: The reward received by the UAV during the training phase: (a) the source task (obstacle-free) and (b) the environment with obstacles

obtained a trained model capable of reaching targets in 3D environment with continuous action space. Then, using the knowledge gathered by the first training, we trained the model to be able to avoid obstacles. Fig. 3.15 shows that the UAV model has converged and reached the maximum possible reward value. During the testing phase, for the obstacle-free environment, the UAV successfully reached its target for the tested cases, 100% success rate for 1000 test case. As for the environment with obstacles, in the case of env1, the UAV successfully

reached its target safely for 84% of the 1000 tested scenarios and in the case of env2, the reached its target safely for 82% of the 1000 tested scenarios.

Table 3.4: Task-completion rate

Scenarios	obstacle free	obstacles env1	obstacles env2
Completion rate	100%	84%	82%

3.6 Summary

In this final chapter, we presented our simulations and results for different use cases. The behavior of the autonomous UAV for the investigated scenarios corroborates the ability of our frameworks to establish efficient employment of UAVs in smart cities applications.

Conclusion and future work

Most of the used UAVs for smart city applications are designed to operate under remote human control or managed via centralized instructions. The aim of this project is to facilitate the transition towards agent-based intelligence in the UAVs themselves, which is trained to learn its working environment and determine its schedule to execute its mission. With semi-supervised machine learning methods, such as reinforcement learning, new decision-making approaches are performed without external intervention. In this study, we frame the development of agent-based intelligence in the context of UAV-based smart city applications. The proposed mechanism considers the characteristics/specifications such as power consumption and battery capacity of the UAV as well as the environment constraints.

In this study, as first step, we developed a generic framework for autonomous UAVs while taking into account their limited battery capacity. The UAVs perform an in-situ decisions via reinforcement learning to determine their activity plans to cover multiple expected and unexpected events. To address this problem, we modeled a reward function that jointly considers the limited battery capacity of the flying units, the time windows of events, and the delays caused by their navigation between events. The aforementioned cost function serves as the basis for the Q-learning approach we later took to solve the problem. The simulation results exhibit the capability of UAVs in learning from the surrounding environment to determine their trajectories in real-time. We obtained results

for different pre-defined scenarios that showed how the UAVs adjust their trajectories in case of emergency events and how they cooperate and substitute each other in case of energy depletion.

In the second step of this study, we developed an efficient framework for autonomous obstacle-aware UAV navigation in urban areas. Using a DDPG-based deep reinforcement learning approach, the UAV autonomously determines its trajectory to reach its assigned static or dynamic destination within a continuous action space. We utilized a transfer learning approach to maximize a reward function balancing between successful navigation and penalized collisions, while minimizing the number of training epochs required for our DDPG approach to converge to an acceptable solution.

Our simulation results demonstrate the capability of UAVs to learn from their environment with the aim of self-improvement of their activity plan and trajectories in real-time. The obtained results investigated different scenarios and showed how the UAVs adjusted their trajectories in case of emergency events and how they cooperated and substituted each other in case of energy depletion. Additionally, the numerical results showed how the UAVs became environmentally aware and smoothly avoided obstacles to safely reach its destinations.

In our future work, we plan to explore the challenges of UAV fleet operation and scheduling in more complex environments such as urban areas with more complex geographic spaces and with the aim of avoiding collisions. In addition, we plan to further investigate solutions to the limited battery capacity of UAVs. After that, we plan to conduct case studies of our approaches on real UAV hardware to demonstrate viability of our approaches in real-life.

Accepted/Submitted publications

[1] **O. Bouhamed**, H. Ghazzai, H. Besbes, and Y. Massoud, *"Q-learning based Routing Scheduling For a Multi-Task Autonomous Agent"* **accepted** *in IEEE 62nd International Midwest Symposium on Circuits and Systems (MWSCAS'19)*, Dallas, TX, USA.

[2] **O. Bouhamed**, H. Ghazzai, H. Besbes, and Y. Massoud, *"A Generic Spatiotemporal Scheduling for Autonomous UAVs: A Reinforcement Learning-based Approach"* **submitted** *to IEEE Open Journal of the Vehicular Technology Society.*

[3] **O. Bouhamed**, H. Ghazzai, H. Besbes, and Y. Massoud, *"Autonomous Unmanned Aerial Vehicle for Wireless Data Collection under Time and Energy Constraints"* **submitted** *to IEEE Systems Journal*.

[4] **O. Bouhamed**, H. Ghazzai, H. Besbes, and Y. Massoud, *"Autonomous UAV Navigation: A DDPG-based Deep Reinforcement Learning Approach"* **submitted** *to IEEE International Symposium on Circuits and Systems (ISCAS'20)*, Seville, Spain.

[5] **O. Bouhamed**, X. Wan, H. Ghazzai, and Y. Massoud, *"A DDPG-based Approach for Energy-aware UAV Navigation in Obstacle-constrained Environment"* **submitted** *to IEEE 6th World Forum on Internet of Things (WF-IoT'20)*, New Orleans, Louisiana, USA.

References

[1] Y. Chen, D. Baek, A. Bocca, A. Macii, E. Macii, and M. Poncino, "A case for a battery-aware model of drone energy consumption," *IEEE International Telecommunications Energy Conference (INTELEC'18), Torino, Italy*, pp. 1–8, Oct. 2018.

[2] K. Kuru, D. Ansell, W. Khan, and H. Yetgin, "Analysis and optimization of unmanned aerial vehicle swarms in logistics: An intelligent delivery platform," *IEEE Access*, vol. 7, pp. 15 804–15 831, Jan. 2019.

[3] F. Mohammed, A. Idries, N. Mohamed, J. Al-Jaroodi, and I. Jawhar, "UAVs for smart cities: Opportunities and challenges," in *International Conference on Unmanned Aircraft Systems (ICUAS'14)*, May. 2014.

[4] C. Luo, S. I. McClean, G. Parr, L. Teacy, and R. De Nardi, "UAV position estimation and collision avoidance using the extended kalman filter," *IEEE Transactions on Vehicular Technology*, vol. 62, no. 6, pp. 2749–2762, July. 2013.

[5] Y. Zeng, R. Zhang, and T. J. Lim, "Wireless communications with unmanned aerial vehicles: opportunities and challenges," *IEEE Communications Magazine*, vol. 54, no. 5, pp. 36–42, May. 2016.

[6] M. Messous, S. Senouci, and H. Sedjelmaci, "Network connectivity and area coverage for UAV fleet mobility model with energy constraint," in *IEEE Wireless Communications and Networking Conference, Doha, Qatar.*, Apr. 2016.

[7] G. Kim, Y. Ong, C. K. Heng, P. S. Tan, and N. A. Zhang, "City vehicle routing problem (city VRP): A review," *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 4, pp. 1654–1666, Aug. 2015.

[8] M. Flood, "The traveling-salesman problem," *Operations Research*, vol. 1, no. 5, pp. 61–75, Feb. 1956.

[9] J. Caceres Cruz, P. Arias, D. Guimarans, D. Riera, and A. Juan, "Rich vehicle routing problem: Survey," *ACM Computing Surveys*, vol. 47, Dec. 2014.

[10] V. V. Panicker and M. Ihsan O, "Solving a heterogeneous fleet vehicle routing model - a practical approach," in *IEEE International Conference on System, Computation, Automation and Networking (ICSCA'18), Pondicherry, India*, July. 2018.

[11] C. Lagos, G. Guerrero, E. Cabrera, A. Moltedo-Perfetti, F. Johnson, and F. Paredes, "An improved particle swarm optimization algorithm for the VRP with simultaneous pickup and delivery and time windows," *IEEE Latin America Transactions*, vol. 16, no. 6, pp. 1732–1740, June. 2018.

[12] K. Sundar and S. Rathinam, "Algorithms for routing an unmanned aerial vehicle in the presence of refueling depots," *IEEE Transactions on Automation Science and Engineering*, vol. 11, no. 1, pp. 287–294, Jan. 2014.

[13] H. Ghazzai, A. Kadri, M. Ben Ghorbel, H. Menouar, and Y. Massoud, "A generic spatiotemporal UAV scheduling framework for multi-event applications," *IEEE Access*, vol. 7, pp. 215–229, Dec. 2018.

[14] H. Ghazzai, H. Menouar, A. Kadri, and Y. Massoud, "Future UAV-based its: A comprehensive scheduling framework," *IEEE Access*, vol. 7, pp. 75 678–75 695, June. 2019.

[15] A. Bahabry, H. Ghazzai, G. Vesonder, and Y. Massoud, "Low complexity algorithms for space-time scheduling of UAVs in smart cities using dimensionality reduction approaches," *International Systems Conference (SysCon 2019), Orlando, FL, USA*, Apr. 2019.

[16] Z. Zhang, J. Wang, J. Li, and X. Wang, "Uav path planning based on receding horizon control with adaptive strategy," in *29th Chinese Control And Decision Conference (CCDC'17), Chongqing, China*, May 2017.

[17] A. Bahabry, X. Wan, H. Ghazzai, G. Vesonder, and Y. Massoud, "Collision-free navigation and efficient scheduling for fleet of multi-rotor drones in smart city," *IEEE International Midwest Symposium on Circuits and Systems (MWSCAS'19), Dallas, TX, USA*, Aug. 2019.

[18] L. Lifen, S., L. Shuandao, and W. Jiang, "Path planning for UAVs based on improved artificial potential field method through changing the repulsive potential function," in *IEEE Chinese Guidance, Navigation and Control Conference (CGNCC'16)*, China, Aug. 2016.

[19] J. Chen, F. Ye, and T. Jiang, "Path planning under obstacle-avoidance constraints based on ant colony optimization algorithm," in *IEEE 17th International Conference on Communication Technology (ICCT'17)*, China, Oct. 2017.

[20] F. Ge, K. Li, W. Xu, and Y. Wang, "Path planning of uav for oilfield inspection based on improved grey wolf optimization algorithm," in *Chinese Control And Decision Conference (CCDC'19)*, China, June. 2019.

[21] T. Zaza and A. Richards, "Ant colony optimization for routing and tasking problems for teams of UAVs," in *UKACC International Conference on Control (CONTROL'14)*, July. 2014.

[22] S. Kato, E. Takeuchi, Y. Ishiguro, Y. Ninomiya, K. Takeda, and T. Hamada, "An open approach to autonomous vehicles," *IEEE Micro*, vol. 35, no. 6, pp. 60–68, Nov. 2015.

[23] N. M. Emanuele Menegatti and B. Yamaguchi, *Intelligent Autonomous Systems*. Springer, Sept. 2015.

[24] J. Deeds, Z. Engstrom, C. Gill, Z. Wood, J. Wang, I. S. Ahn, and Y. Lu, "Autonomous vision-based target detection using unmanned aerial vehicle," in *IEEE International Midwest Symposium on Circuits and Systems (MWSCAS'18)*, Windsor, ON, Canada, Aug. 2018.

[25] M. Nazari, A. Oroojlooy, L. V. Snyder, and M. Takác, "Deep reinforcement learning for solving the vehicle routing problem," *CoRR*, vol. abs/1802.04240, May. 2018.

[26] E. Camci and E. Kayacan, "Waitress quadcopter explores how to serve drinks by reinforcement learning," in *IEEE Region 10 Conference (TENCON'16)*, Singapore, Nov. 2016.

[27] N. Imanberdiyev, C. Fu, E. Kayacan, and I. Chen, "Autonomous navigation of UAV by using real-time model-based reinforcement learning," in

14th International Conference on Control, Automation, Robotics and Vision (ICARCV'16), Phuket, Thailand, Nov. 2016, pp. 1–6.

[28] R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*, ser. Adaptive Computation and Machine Learning series. MIT Press, 2018.

[29] R. Bellman, *Dynamic Programming*, ser. Dover Books on Computer Science. Dover Publications, 2013.

[30] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, "Playing atari with deep reinforcement learning," *CoRR*, vol. abs/1312.5602, 2013.

[31] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, Feb. 2015.

[32] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning." in *ICLR*, 2016.

[33] T. Lillicrap, J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *CoRR*, 09 2015.

[34] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Transactions on Wireless Communications*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.

[35] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569–572, Dec. 2014.

[36] M. Khan, B. A. Alvi, E. Safi, and I. Khan, "Drones for good in smart cities:a review," *International Conference on Electrical, Electronics, Computers, Communication, Mechanical and Computing (EECCMC'18), Tamil Nadu, India*, Jan. 2018.

[37] H. Ghazzai, A. Kadri, M. Ben Ghorbel, H. Menouar, and Y. Massoud, “A generic spatiotemporal UAV scheduling framework for multi-event applications,” *IEEE Access*, vol. 7, pp. 215–229, Jan. 2019.